Set aside

These passages were taken out of *The Thiele Machine: No Free Insight* and are kept here. Numbered theorems and sections cited below and not found here are the book's.

Contents

1 Constructing the quoting operation 3
- 1.1 Numerals and syntax as data 3
- 1.2 Recursion without running under a lambda 4
- 1.3 Quoting numerals 5
- 1.4 Quoting terms 5

2 From "A record and a toll" 7

3 From "Why the toll isn't arbitrary" 14

4 From "Structural entitlement" 19
- 4.1 The count, exactly 19
- 4.2 When the run does the narrowing 22

5 From "Records over any base: the latch" 24
- 5.1 Bases from outside, and a relation I have to be honest about 24

6 From "The shadow, exactly" 28
- 6.1 How much gets destroyed, on average 28

7 From “The diagonal” 31

8 From “What established means” 36
8.1 The guy who got closest 36

9 From “Five small models” 37
9.1 One example, three levels 38

10 From “The pointer thesis” 40
10.1 What’s proved 40
10.2 What isn’t proved, and must stay that way on the page 45
10.3 What the thesis asks of the examples 45

11 From “The counter against physics” 47
11.1 The calibration is a unit conversion 47
11.2 The calorimeter protocol, as a specification 49
11.3 What this section is 53

12 From “The claims and their conditions” 54
12.1 The diagonal and the worked models 54

13 Methodology note 55

14 Rows of the claim ledger for the passages above 56

15 From the plain-text statement 60

16 From the README 63

17 From the plain-text statement and the README, second pass 64

18 Further reading 67
18.1 Structure, information and knowledge 67
18.2 Cost, records and thermodynamics 68
18.3 Computation, self-reference and checking 72
18.4 Environmental records 75

CHAPTER 1

Constructing the quoting operation

The proof in Section 14 uses a program that takes the code of a term and returns the code of that code. Here's how to build it.

Draw the instruction "add one" on a card. Now make a second card describing the first card's marks, rather than carrying out the addition. The count has not risen: you have described an instruction. Describe the second card in turn and you have a description of a description. Quoting does that second job mechanically. In the language below, take the variable of index 0. Its code is a tree with three lambda nodes, then an application of variable 2 to the numeral for 0. Quoting that code builds a description of those three lambdas and that application; it does not look up variable 0 or execute the term it describes.

Nothing in the operation lets a program inspect itself by magic: its input is an ordinary tree encoded as a closed lambda term.

The construction follows Forster and Smolka (2017), whose construction for L has the same three syntax builders on Scott-encoded children, an auxiliary procedure for numerals, and a quote procedure Q with the same specification, defined by the same recursion on codes; the self-application that uses Q in Section 14 is their fixed-point lemma, after Barendregt's (1984) proof for the full lambda calculus. What follows is my rebuilding of it, with the reductions written out.

1.1 Numerals and syntax as data

For readability, the formulas below give names to bound variables. Renaming a bound variable changes nothing; the underlying syntax uses the de Bruijn indices of Definition 7.1. Applications associate to the left. Write $X \Downarrow Y$ here for a finite sequence of the weak call-by-value steps ending in the value Y.

Scott numerals are the following closed values:

$$\overline{0} = \lambda z.\lambda s.z, \qquad \overline{n+1} = \lambda z.\lambda s.s\,\overline{n}.$$

So a numeral selects either the supplied zero case or the supplied successor case, passing its predecessor to the latter. The cases below are values, so choosing one does not prematurely run the other.

The three syntax constructors, operating on already encoded children, are

$$\begin{aligned}\text{mkvar} &= \lambda n.\lambda v.\lambda a.\lambda l.(v\,n),\\ \text{mkapp} &= \lambda p.\lambda q.\lambda v.\lambda a.\lambda l.a\,p\,q,\\ \text{mklam} &= \lambda p.\lambda v.\lambda a.\lambda l.l\,p.\end{aligned}$$

Each builder is a term of the language L of Definition 7.1. For a variable of index n, an application and an abstraction, respectively, define their codes by the following resulting values:

$$\begin{aligned}\ulcorner \#n \urcorner &= \lambda v.\lambda a.\lambda l.(v\,\overline{n}),\\ \ulcorner p\,q \urcorner &= \lambda v.\lambda a.\lambda l.a\ulcorner p\urcorner\ulcorner q\urcorner,\\ \ulcorner \lambda p \urcorner &= \lambda v.\lambda a.\lambda l.l\ulcorner p\urcorner.\end{aligned}$$

In particular, $\text{mkvar}\,\overline{n}$, $\text{mkapp}\ulcorner p\urcorner\ulcorner q\urcorner$ and $\text{mklam}\ulcorner p\urcorner$ reduce to these three codes. A code is closed even when the term it describes has a free variable: the index of that variable is stored as a numeral. It is data inside the code, and the code itself has no free variable.

1.2 Recursion without running under a lambda

The quoting procedure will have to call itself on the smaller pieces of a description. There is a catch: this language works out an argument before handing it to a function. Hand over the recursive calculation itself and it starts calculating at once, before the receiving function has decided whether it needs it. That can send you straight back into the same calculation forever.

Instead, hand over a recipe waiting for an ingredient. In this language a lambda is such a recipe, and it counts as a finished value even though its body has not run. The receiving function can keep the recipe and call it when it reaches a smaller piece. In the expression below, the little recipe $\lambda a.x\,x\,f\,a$ is the part doing that job. The letter a names the argument it is waiting for; f names the work to do once the recipe and the current input have been handed over.

$$W = \lambda x.\lambda f.\lambda v.\ f\,(\lambda a.x\,x\,f\,a)\,v, \qquad \text{rec}(F) = \lambda v.W\,W\,F\,v.$$

When F and u are closed values, beta reduction gives

$$\text{rec}(F)\,u \longrightarrow W\,W\,F\,u \longrightarrow^{*} F\,(\lambda a.W\,W\,F\,a)\,u = F\,(\text{rec}(F))\,u.$$

The recursive argument is a lambda, so it's already a value. It starts work only when F applies it to an argument. That is the whole reason this construction survives the call-by-value rule.

1.3 Quoting numerals

Let $E_j = \ulcorner \#j \urcorner$ for $j = 0, 1, 2$, and let $Z = \ulcorner \overline{0} \urcorner$, obtained directly from the finite syntax of $\overline{0} = \lambda\lambda.1$. Write $\mathrm{mklam}^k(X)$ for k nested applications of the syntax builder mklam to X; it is an abbreviation for constructing k lambda nodes, and the language gains no new operation from it.

Define

$$F_N = \lambda r.\lambda n.\ n\, Z\, (\lambda k.\mathrm{mklam}^2(\mathrm{mkapp}\, E_0\, (r\, k))), \qquad Q_N = \mathrm{rec}(F_N).$$

Follow the two branches. Zero chooses the already built description Z. A number one larger than n chooses the other branch and hands it the predecessor n. That branch first asks the same procedure to describe the predecessor. Then it puts the description inside an application of variable zero and wraps two lambdas around it. Those are precisely the marks in the numeral for $n + 1$.

Zero therefore works. If the procedure describes n correctly, the wrapping just described makes it describe $n + 1$ correctly. Starting at zero and taking one more step proves the claim for every natural number. Every call on a positive number asks about a smaller one, so it reaches zero and finishes. In the notation for termination and returned value, the result is $Q_N\, \overline{n} \Downarrow \ulcorner \overline{n} \urcorner$.

Do the smallest case by hand. Call the successor case G. Then $Q_N\, \overline{1}$ steps to $F_N\, Q_N\, \overline{1}$ and on to $\overline{1}\, Z\, G$; the numeral hands its predecessor $\overline{0}$ to G, the recursive call $Q_N\, \overline{0}$ comes back as Z, and the builders return $\mathrm{mklam}(\mathrm{mklam}(\mathrm{mkapp}\, E_0\, Z))$. That is the code of $\lambda\lambda.(\#0\, \overline{0})$, which is $\overline{1}$.

1.4 Quoting terms

For a proposed recursive call r, define three handlers:

$$\begin{aligned} H_V &= \lambda n.\mathrm{mklam}^3(\mathrm{mkapp}\, E_2\, (Q_N\, n)), \\ H_A(r) &= \lambda p.\lambda q.\mathrm{mklam}^3(\mathrm{mkapp}\, (\mathrm{mkapp}\, E_1\, (r\, p))\, (r\, q)), \\ H_L(r) &= \lambda p.\mathrm{mklam}^3(\mathrm{mkapp}\, E_0\, (r\, p)). \end{aligned}$$

Now set

$$F_Q = \lambda r.\lambda t.t\, H_V\, (H_A(r))\, (H_L(r)), \qquad Q = \mathrm{rec}(F_Q).$$

The handlers are lambdas and so are values. The Scott code of the input chooses one of them and hands it the encoded children. Start at the leaves and build outward.

At each node the handler builds the syntax of that node's encoding: $Q\ulcorner t\urcorner \Downarrow \ulcorner\ulcorner t\urcorner\urcorner$. There are three kinds of node to consider:

- For $t = \#n$, the first handler constructs three lambda nodes over an application of variable 2 to the quoted numeral. That is the syntax of $\lambda\lambda\lambda.(\#2\,\overline{n})$, the code of $\#n$.
- For $t = p\,q$, quote the two children first. The second handler constructs three lambdas over variable 1 applied to those two codes. That is the syntax of the application code.
- For $t = \lambda p$, the last handler constructs three lambdas over variable 0 applied to the quoted child code. That is the syntax of the abstraction code.

Each recursive call works on a proper child of the finite input tree, or on a smaller numeral. No chain of calls can keep descending through a finite tree or decreasing a natural number forever. Each builder takes finitely many beta steps, so the construction finishes and returns the code just described.

The mkapp used in Section 14 is this builder. With Q and that builder written out, every piece of the self-referential term there is on the page.

CHAPTER 2

From “A record and a toll”

Take an ordinary computing machine and put a lamp and tally beside it. Let computing run as before, leaving both alone. A separate mark leaves the computing state fixed, adds one to the tally and turns the lamp on when its named event holds. Forget the lamp and tally and the old computation is still there, step for step. Below I build that box, compare different ways to implement it, and ask exactly which differences its questions require. The old machine may already have prices or a reading of its own; this construction supplies the new lamp and its zero/one account. With effective rules an ordinary computer can store all three parts.

What the next move could tell us

Put two boxes beside each other and read everything the task lets you read. Suppose the answers agree. That is not yet enough to put the boxes in the same pile. Press the same button on each. Perhaps one lamp now comes on and the other stays dark. The difference mattered, even though the first reading missed it.

Keep two states together only when every permitted finite continuation gives the same required answer. Include doing nothing: the current answer counts too. This tells us which differences the task can afford to forget.

Theorem 2.1 (Keep what a later question can separate)**.** For a fixed deterministic machine, permitted instructions and required observation, equality of observations after every finite continuation is the unique coarsest equivalence that preserves the current observation and is stable under every instruction. A deterministic implementation preserving those steps and answers cannot identify two states separated by such a continuation. □

Proof. Agreement with oneself, reversal of an agreement, and joining two agreements all hold one continuation at a time, so this is an equivalence. Doing nothing checks the current observation. To check stability after a move, put that move in front of each continuation. If two states were already grouped by any observation-preserving, stable relation, apply its stability once for every move in a continuation,

then read the answer. Those states agree on every continuation and belong to our equivalence. Our grouping therefore contains every permitted grouping. Finally, if an implementation put two separated states in one full state, the same continuation from that state would have to produce two different answers. Determinism gives only one. ■

Change the questions and the piles can change. A box that toggles between two positions needs two distinguishable states if I ask its position. If my only required answer is always “ready”, and every later answer is also “ready”, those positions can share one description. The task tells me what has to survive.

The picture, the lamp and the tally

Take an ordinary computing box. Put a lamp beside it and a tally beneath it. A computing button moves the box by its old rule, costs zero, and leaves the lamp and tally alone. A mark button leaves the box where it is, adds one to the tally, and turns the lamp on if its named event holds there. Once on, the lamp stays on. Start with the lamp down and the tally at zero.

Those instructions tell us what to do without telling us how to build the box. One builder may keep every button press on a tape. Another may keep an extra turn counter. I require each builder to show me the same three things: the computing picture, the lamp and the exact tally. Its computing moves must follow the base rule through that picture. Its marks must leave the picture fixed and latch exactly the named event. Its prices are zero for computing and one for marking; its reading obeys the toll and permanence; its tally increases by exactly the charge on every move.

□ **Theorem 2.2** (Every such box has the same three-part description)**.** Fix those base rules, mark events and accounting requirements. Every implementation has exactly one map to the product of base state, Boolean reading and natural tally that preserves the three readings. That map preserves every instruction and every finite run. From a start with lamp down and tally zero, it maps the reachable implementation states onto all reachable states of the product lift with the matching base start.

Proof. Read the picture, lamp and tally, and write them down in that order. There is no other choice for a map required to preserve those three readings. Press a computing button. The picture follows the base rule. The lamp stays put by the toll and permanence, and the tally stays put because the charge is zero. Press a mark instead. The picture stays put, the lamp takes its old value or the event’s yes, and

the tally rises by one. These are exactly the two rules of the product lift. Repeat for every move in a finite run.

The clean starts agree in all three readings. Any reachable product state has some finite list of buttons reaching it. Press that same list on the implementation. The step argument makes its three readings the desired product state. Every reachable product state therefore has a reachable implementation state above it. ■

Here are the three stages. First write a list of buttons. Then run it on the builder's box. Then read the picture, lamp and tally. You get the same result as running the list directly on the three-part description. Written as arrows, after we have done it:

$$\text{button lists} \longrightarrow \text{implementation states} \longrightarrow \text{picture, lamp, tally.}$$

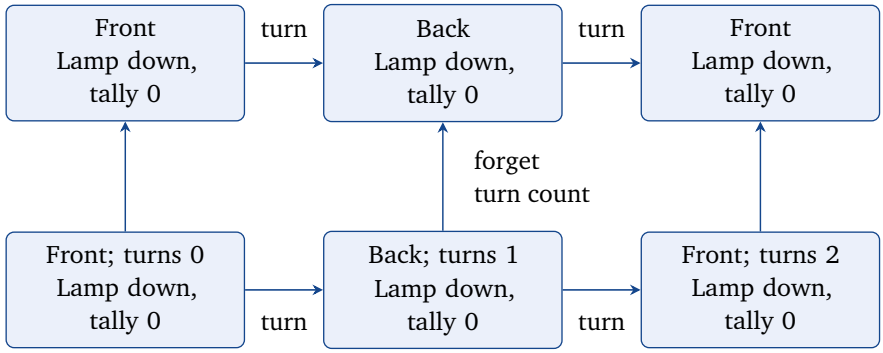
The first arrow has only one possible meaning once the start and buttons are fixed. The empty list gives the start; appending a button takes one more step. This is the free history construction. The second arrow can forget history. Its direction matters.

Turn a box halfway around with a free button. Turn it again. It is back where it started, with the same lamp and tally. Now give another builder's box a counter that goes up on every turn, without charging for it. After two turns its counter is two larger. It meets all the requirements above: the extra counter is separate from the tally of charges.

Proposition 2.3 (Forgetting the turn count goes one way)**.** Even on reachable states, the three-part lift need not have a step-preserving map into every implementation satisfying its contract. □

Proof. In this example two turns return every product state to itself. A map preserving turns would have to send it to an implementation state that also returns after two turns. No such state exists there: its extra counter has increased by two. Forgetting that counter works in the other direction and preserves both turning and marking. ■

There is an exact way to tell when the reverse map would work. Whenever two button lists reach the same source state, they must also reach the same target state. Otherwise the map would have to send that one source to two targets. Supply one access list reaching each source state, and this condition is sufficient: run its access list on the target. Any other list reaching that source gives the same target by the condition. The empty list preserves the start, appending a button preserves a step, and reachability leaves no other possible map. The turn counter fails exactly this test: the empty list and two turns agree in the source and disagree in the target.



The top row returns to the same full state. The bottom row does not. Forgetting the turn count respects each turn; recovering it from the top row cannot.

□ **Theorem 2.4** (The smallest description that answers these questions)**.** Under the preceding contract, two implementation states give the same three readings after every common finite continuation exactly when their current three readings agree. If every two distinct reachable implementation states can be separated by such a continuation, projection is a bijection onto the reachable product states. It preserves starts and steps and is the unique start- and step-preserving map there.

Proof. If the current readings differ, read them now. If they agree, the two product states are equal. Equal states obeying the same rule give equal next states; repeat for every move. Projection preserves those moves, so the implementation’s readings agree throughout too.

Now require every distinct reachable pair to be separable. Two states projecting to the same triple could never be separated, so they must be one state. Projection is therefore one-to-one; the previous theorem makes it onto the reachable product states. Any start- and step-preserving map is fixed on a reachable state by a list reaching it: map the start, then follow the list. It must agree with projection there. ■

The same reasoning explains uniqueness more generally. Take two reachable machines with the same answers to every button list from their starts, and suppose each keeps no two states that all future answers fail to separate. Supply an access list for each state of the first. Run that list on the second. If two lists reach the same first state, append any continuation: their answers agree on the first machine, hence on the second. Separation in the second makes their endpoints equal. The map is well-defined and preserves steps and outputs. Reachability makes it onto; separation in the first makes it one-to-one. Following an access list forces uniqueness.

For our box, that smallest description keeps the tally because I asked for the tally. Stop asking for it and repeated failed marks may become indistinguishable. Ask for

every past button and the extra tape starts to matter. With a clean start and an event that never holds, no reachable state has its lamp up. Counting unreachable lamp-up states would answer a different question. “Smallest” here means no two retained states are indistinguishable by the required future readings, including on an infinite machine.

This way of grouping states is the familiar behavioral equivalence behind minimal automata. The initial history construction and final observation construction are treated in Rutten’s *The Method of Coalgebra*, Chapters 2 and 4 and Section 8.6.[1] Here the box tells us exactly which observations and operations the lift preserves. Choosing the event, the window and the accounting is where the application enters. Once they are fixed, the argument above determines which implementation differences can be forgotten.

What can choose the question?

Take a dial with two faces and one button. Each press turns it to the other face. For this question, supply no starting face, no lamp and no names with an outside meaning. Draw its two arrows. Now exchange the names on the two faces. The arrows describe exactly the same machine.

Suppose those arrows alone are meant to tell us which face deserves yes. A rule that does not depend on our choice of names must give the exchanged face the same reading. So both faces get the same answer. A nonconstant reading cannot be selected that way. Give one face a role, such as the loaded start, a completed job or a passed check, and I have supplied more to work with. The role has to be stated and defended. Renaming the arrows did not supply it.

The argument is two lines: the exchange takes the first face to the second; respecting that exchange makes their readings equal. Those are all the faces. It does not make every machine’s reading constant. It rules out a nonconstant reading selected from the transition structure alone on every machine. [2]

That fixes what the uniqueness claim can say. The box, lamp and tally have a unique observable realization under the contract already proved. The arrows by themselves do not choose that contract. Adding the word “intrinsic” cannot do the choosing for us.

[1] Jan Rutten, *The Method of Coalgebra* (2019), `https://www.cs.cornell.edu/courses/cs4810/2025sp/Handouts/Rutten.pdf`.

[2] Checked in `coq/audit/IntrinsicBoundary.v`: `toggle_symmetry_forces_constant_reading` and `no_nonconstant_renaming_invariant_toggle_reading`.

Does the picture keep enough of the device?
Put a lamp on a box and a hidden switch inside it. The button copies the hidden switch to the lamp. Start with the lamp dark in both boxes, and put the hidden switch down in one, up in the other.

Lamp before	Hidden switch	Lamp after the button
Dark	down	dark
Dark	up	lit

A description that keeps only the lamp has one starting state, dark, and is asked for two different next answers to the same button. It cannot do it. Keep the hidden switch too and the problem disappears. This is a reason the additional distinction is necessary for this task: the next press can expose it. It is not a reason every task needs that switch. [3]

There is a complete test for this kind of description. First write down the underlying configurations, the permitted buttons, every permitted successor, the picture being retained, and the answers the task requires. This is the operational model of the device. Then check two things. Configurations with the same picture must give the same required answer now. And the same button, from any configurations with that picture, must leave the same next picture, for every permitted successor.

Both conditions are necessary. An answer decoder has only the picture to read. A deterministic next-picture rule has only the picture and button. Neither can return two different results on one input.

They are sufficient too, with explicit choices. Supply a representative configuration for each picture and a permitted successor for each button at each configuration. Decode a picture by reading its representative. Compute its next picture by pressing the button on that representative and taking the chosen successor. The two conditions ensure that every other configuration and every other permitted successor agrees. Repeat that argument at each step: all finite button lists preserve the required answers. On a finite listed model this can be checked against its complete table. For arbitrary state types the choices and their effectiveness are separate data; the proof does not manufacture a search procedure.

This test allows several successors of an underlying configuration.[4] If two of them give different required answers, no exact deterministic encoding of that same configuration and button can represent both. [5] A probability distribution over outcomes, or a finer state including a chosen input or seed, is a different contract and must be described as one.

[3] `hidden_bit_blocks_visible_only_dynamics`, `coq/audit/ObservationContract.v`.

[4] `operational_adequacy_iff`, `coq/audit/ObservationContract.v`.

[5] `branching_answers_exclude_exact_deterministic_encoding`, `coq/audit/ObservationContract.v`.

The test starts with a model. Establishing that a real apparatus follows that model still needs evidence about the apparatus: preparation, controls, readout and the range over which the description is claimed. Passing the mathematical test does not supply those measurements.

How long did we watch the lamp?
Watch two boxes for three ticks. Both start dark and light on the first tick. Both stay lit on the second and third. One stays lit forever. The other has a counter inside that turns it off on the fourth tick.

Tick	Enduring lamp	Delayed reset
0	dark	dark
1	lit	lit
2	lit	lit
3	lit	lit
4	lit	dark

The first three ticks do not tell the boxes apart. Pick any longer finite watch and I can move the reset one tick beyond it. Both boxes can pay one unit per tick and satisfy the basic toll. Only the first has permanence. The permanent interface excludes the second by its rule, not by pretending that the finite observation already excluded it.

The construction works for every finite horizon.[6] It is not a prediction that every real record fails. It shows why a finite run alone cannot establish an unrestricted forever claim. A proof from an adequate mechanism, or an explicitly bounded reliability claim supported by measurements, has work to do beyond reporting that the lamp stayed lit while we watched.

[6] `finite_observations_do_not_establish_permanence`, `coq/audit/IntrinsicBoundary.v`.

CHAPTER 3

From “Why the toll isn’t arbitrary”

There is also a limited statement about how long a noisy record lasts, using a different quantity: total entropy production during relaxation. Take a device with no outside hand: a random process hopping among finitely many states at rates that never change, with a reverse for every hop and local detailed balance, able to get from any state to any other. Require at least one up-state and one down-state. Left to run, it settles into a long-run spread π. Measure each up or down stretch from its entrance, with entrances sampled in the stationary process, and call the mean lengths T_{up} and T_{down}. Start the device in a known no-state s. The total entropy it produces over relaxation is at least $\ln(1 + T_{\mathrm{up}}/T_{\mathrm{down}})$, in units of k_B.

First compare the starting spread with the spread the device settles into. A state that is common at the start but rare in the settled spread is a large mismatch. For each state, take the ratio of those two chances, take its natural logarithm, and average using the starting chances. With p the starting spread, that number is

$$D(p\|\pi) = \sum_{x:p(x)>0} p(x) \ln \frac{p(x)}{\pi(x)}.$$

It is called relative entropy. Here every $\pi(x)$ is positive, so the ratios are available. The number is nonnegative: apply $\ln y \le y - 1$ to $\pi(x)/p(x)$, multiply by $p(x)$, and add. The right side is at most zero, since the included π chances total at most one and the p chances total exactly one. Negating that comparison gives $D(p\|\pi) \ge 0$.

For a known start s, all the starting chance is at s. There is only one term: $D = \ln(1/\pi(s)) = -\ln \pi(s)$. A state with settled chance one tenth therefore starts with mismatch $\ln 10$. Since s is a no-state, its settled chance cannot exceed the total chance of all no-states. Its mismatch is at least $-\ln \pi(\mathrm{no})$.

The physical step in the argument is the fixed-rate entropy-production split of Massimiliano Esposito and Christian Van den Broeck (2010). In that setting the total entropy produced is at least the decrease in this relative entropy. Once the spread has settled to π, the relative entropy left is zero. So the initial mismatch supplies the lower bound for the whole relaxation.

Now read the same number from the record of up and down stretches. In a long settled run, each entry into YES is followed by an exit; apart from the two ends of the record, there are equally many up-stretches and down-stretches. Total time up is the number of up-stretches times their mean length, and likewise for time down. Dividing gives

$$\frac{\pi(\text{up})}{\pi(\text{no})} = \frac{T_{\text{up}}}{T_{\text{down}}}.$$

The two chances add to one. If the ratio on the right is r, the no-share is $1/(1+r)$, and its negative logarithm is $\ln(1+r)$. That gives the stated bound. The settling and mean-length arguments below justify passing from a long record to these limiting quantities.

For a large up-to-down ratio, doubling that ratio raises this lower bound by about $k_B \ln 2$. Making both stretches longer by the same factor leaves it unchanged. Across a family of devices whose ratio tends to infinity, the bound diverges; no single finite irreducible device in this class keeps its flag up forever. This is a relaxation bound, not a heat charge on each flip. It uses neither a thermodynamic reverse-path construction nor time-symmetric control, so it does not identify the quantities in those other results. It also leaves the known-input case of the deterministic entropy-loss bound where it was: this noisy evolution and its total entropy production are a different question. The entropy is added up over one whole relaxation, from the known no-state s until the device has settled, and local detailed balance is what ties it to heat exchanged with the reservoirs. Total entropy production includes the change in the device's entropy too; it is not heat alone. It also isn't the permanence of the finite-state theorems on a noisy device: on this device the flag comes back down, and T_{up} and T_{down} are average stretches.

The core of it for a chain that ticks, worked out, runs as follows in the case where the long-run spread π gives every state a positive chance and satisfies detailed balance: in the long run as much weight flows from x to y in one step as from y to x, $\pi(x)P(x,y) = \pi(y)P(y,x)$, where $P(x,y)$ is the chance of stepping from x to y. Count the entropy produced in one step as the change in the device's own entropy plus what goes out to the surroundings, which for a hop from x to y is $\ln(P(x,y)/P(y,x))$. Detailed balance turns that ratio into $\pi(y)/\pi(x)$, and the sum collapses: the entropy produced in one step is exactly the drop in relative entropy to π. It's never negative, because a step that leaves π where it is can't make any other spread harder to tell from π (that's the data-processing inequality, a standard fact of information theory). Add the steps up and the sum telescopes. A known start s has relative entropy $-\ln \pi(s)$, so the entropy produced in the first T steps is $-\ln \pi(s)$ minus the relative entropy left at step T: it never exceeds $-\ln \pi(s)$, and it reaches it in the limit exactly when that remainder goes to zero. Next, $-\ln \pi(s) \geq -\ln \pi(\text{no})$,

because s is one of the no-states and so carries at most their total chance. The flow from the no-states to the yes-states equals the flow back, because in the long run the weight on the yes-states doesn’t change, so what flows in equals what flows out. Last, take each mean stretch as its mass over that flow; then $T_{\text{up}}/T_{\text{down}} = \pi(\text{up})/\pi(\text{no})$, and since the two chances add to one, $-\ln \pi(\text{no}) = \ln(1 + T_{\text{up}}/T_{\text{down}})$.

Try the stretches with a bag of ten counters: nine marked YES and one NO. Draw, record the answer, replace the counter and mix again. Each draw gives YES with chance $9/10$, whatever the last answer was. A YES stretch ends on the next draw with chance $1/10$, so its mean length is ten draws: the mean L obeys $L = 1 + (9/10)L$. A NO stretch similarly averages $10/9$ draws. Their ratio is nine, just like the ratio of YES to NO counters. In a long record, count all the YES entries and divide by the number of YES stretches; that is the measured mean stretch length. Divide both counts by the total number of draws and you have mass divided by flow.

For this rule the long-run flow from NO to YES is $(1/10)(9/10) = 9/100$ per draw, and the reverse flow is the same. A start known to be NO has relative entropy $-\ln(1/10) = \ln 10$ against the long-run spread; after one draw the spread is already $1/10, 9/10$, so none of that relative entropy remains. This bag shows the probability accounting. The entropy-production interpretation still needs the stated physical model; shaking counters in a bag does not measure that heat.

Two things need establishing for a general device: that it actually settles, and that “mean stretch length” means what it sounds like. I’ll say what each says, and why it holds.

Settling comes first. A chain doesn’t have to settle. It can get stuck in a corner, or cycle round forever. What rules that out here is a condition named for Wolfgang Doeblin, a German-born French mathematician, son of the novelist Alfred Döblin, who took his doctorate at the Sorbonne in 1938 for work on chains like this one and died in June 1940, at twenty-five. In words: from wherever the device is, its next step lands on each state with at least some fixed fraction δ of that state’s long-run chance. No state is ever out of reach in one step, and none is favoured so heavily that the others starve. Under that condition each step shrinks the distance between where the device is and where it’s going by the factor $1 - \delta$ (the distance here is the total of the differences in chance, state by state). To see why, split each step in two: with chance δ, forget where the device was and draw a fresh state from π; otherwise, move by whatever is left over, and the condition is exactly what makes the leftover a genuine set of chances. The fresh draw puts the device’s spread and π in the same place, so only the other share, $1 - \delta$, can keep them apart, and no step pushes two spreads further apart than they were. So the distance dies off geometrically and the spread converges to π at every state. The payoff for the bound is that the entropy produced from the known start stays under $-\ln \pi(s)$, and it also gets there: after T steps it falls short by at most $(4/\pi_{\min})(1 - \delta)^{2T}$, where $\pi_{\min}$ is the smallest long-run

chance of any state. The shortfall is the relative entropy left, and $\ln y \le y - 1$ once more puts that under the sum of $(p(x) - \pi(x))^2/\pi(x)$, which is at most the square of the distance over $\pi_{\min}$; the distance starts at no more than 2 and shrinks by $1 - \delta$ a step. So $-\ln \pi(s)$ is the exact total entropy production of the whole relaxation, in units of k_B, and the bound $-\ln \pi(s)$ is reached.

For rates satisfying detailed balance with π, the same equality holds when the device hops at random moments, with no clock ticking. Write out how fast the relative entropy to π changes and pair each hop with its reverse. Detailed balance makes each pair's share minus (the flow one way minus the flow back) times the logarithm of their ratio, and that is exactly the pair's share of the rate entropy is produced; a difference times the logarithm of the same ratio is never negative. So at every instant the relative entropy falls at exactly the rate entropy is being produced, and that rate is never negative; add the rate up between two times and you get exactly the drop in relative entropy between them. For settling, Doeblin's condition becomes a condition on rates: the rate of hopping from any state x to any state y is at least c times $\pi(y)$, for some fixed $c > 0$. Split the rates in two again: a part that, at rate c, forgets where the device was and redraws it from π, and the rest, which still satisfies detailed balance. The production rate adds over the two parts, because with detailed balance it's the rates times quantities that don't depend on the rates. The rest produces nothing negative, and the redraw part alone produces at least c times the relative entropy still left. So relative entropy falls at least as fast as e^{-ct}. One wrinkle needs its own line. A known start puts all its weight on one state and none anywhere else, and the logarithms in the relative entropy go bad at zero. The relative entropy is still continuous at the starting instant, because each of its terms has the form $u \ln(u/\pi)$, and $u \ln u$ goes to zero as u does. So adding up the production from the start out to infinity gives exactly $-\ln \pi(s)$.

I should say what Doeblin's condition costs. It asks for every hop to be possible directly, in one step or at a positive rate. A device whose states reach each other only through other states can still settle, and the standard theory of these chains says when: always, for a device that hops at random moments and can get from any state to any other; and for a chain that ticks, as long as it also doesn't march round a cycle. Those devices meet the condition for some fixed stretch of time or number of steps instead of one, and the same argument runs on blocks of that length, because the relative entropy never goes up in between.

Then come the stretches. The bound used the fact that the long-run chances of up and down stand in the ratio of the mean stretch lengths, and I took the mean length of a stretch with the flag up to be the long-run weight of the up-states divided by the flow into them. For a chain that ticks, that's right for the following reason. Start each stretch the way the flow starts them, count the steps until the chain leaves the up-states, and the average is exactly $\pi(\text{up})$ over the flow. Look at the long-run

weight that has been up at each of the last $k+1$ ticks. It splits in two: the part whose stretch began exactly k ticks ago, which is the flow times the chance that a fresh stretch lasts more than k ticks, and the part that was up one tick earlier still. Add those splits from $k=0$ up to some K and the sum telescopes: $\pi(\text{up})$ is the flow times the sum of those chances, plus the weight that has been up for more than K ticks. The sum of the chances that a stretch lasts more than k is its mean length. Doeblin’s condition makes the leftover die: from any up-state the next step lands in the down-states with chance at least $\delta\,\pi(\text{down})$, so the weight that has stayed up for K ticks is at most $(1-\delta\,\pi(\text{down}))^K$.

With the stretches measured in seconds, when the device hops at random moments, it comes out the same, by a different route. Ask, for each up-state, how long on average the device takes to leave the up-states from there. Call that its leaving time. The leaving times are finite: from every up-state the device hops into the down-states at a total rate of at least $c\,\pi(\text{down})$, so the chance a stretch is still running at time t falls at least as fast as $e^{-c\,\pi(\text{down})\,t}$, and no leaving time is more than one over $c\,\pi(\text{down})$. And they obey one plain rule, which you get by asking what happens in the first instant: from any up-state, the rate out of it times its leaving time, minus the rates into the other up-states times their leaving times, comes to exactly one (per second times seconds is a pure number). Average the leaving times over the way the flow starts stretches, multiply by the flow, and the rule plus the balance of the steady spread (as much weight flows into each state as out of it) rearrange the sum into exactly $\pi(\text{up})$, so in seconds too the mean stretch is its mass over the flow. Both of the bound’s premises are proved now, in steps and in seconds, under Doeblin’s condition.

CHAPTER 4

From “Structural entitlement”

4.1 The count, exactly

I’ll set out the words and then the count, for any system with Section 8’s counter. A *list* is a finite list of candidates, repeats allowed, and $|\Omega|$ is the length of the list Ω, repeats counted. $\lceil \log_2 x \rceil$ is the base-two logarithm of x rounded up: the smallest $k \in \mathbb{N}$ with $2^k \geq x$. At $x = 0$ it is 0, since $2^0 \geq 0$. And $a \mathbin{\dot{-}} b$ is subtraction on whole numbers that stops at zero, so $3 \mathbin{\dot{-}} 5 = 0$.

Definition 4.1 (Decision tree)**.** A *decision tree* is a leaf, or a branch with two subtrees. Its depth is 0 for a leaf and one more than the deeper subtree for a branch; its leaf count $L(T)$ is 1 for a leaf and the sum of the two counts for a branch. Nothing is written on a tree: no question, no answer, no candidate.

Lemma 4.1 (A tree is no wider than two to its depth)**.** For every decision tree T, $L(T) \leq 2^{\mathrm{depth}(T)}$, and so $\lceil \log_2 L(T) \rceil \leq \mathrm{depth}(T)$. □

Proof. Start at the root. At the first level there can be at most two places, at the second at most four, and each further level at most doubles the places available. To count leaves that stop early as well, extend each such leaf with branches until all tips reach the tree’s greatest depth. This only adds leaves. At depth d the filled-out tree has 2^d tips, so the original tree has at most that many leaves. For depth zero there is just the root, which is one leaf.

Thus d bits are enough to number all the leaves. The least number of bits that will do, $\lceil \log_2 L(T) \rceil$, cannot be greater than d. ■

Nothing about any machine is in the definition or the lemma. They’re about trees, a leaf or a branch, and nothing else.

□ **Theorem 4.2** (Paid narrowing, counted)**.** Take a system with a counter μ and a price list c that obey conservation (Theorem 8.2): each step raises μ by exactly its charge. Let Ω and Ω' be lists with $|\Omega'| > 0$, let T be a decision tree with $|\Omega| \le L(T) \cdot |\Omega'|$ (the tree *covers* the narrowing), and let t be a run from s ending at s' in which at least $\mathrm{depth}(T)$ steps are charged at least one each. Then

$$|\Omega| \;\le\; 2^{\mu(s')-\mu(s)} \cdot |\Omega'|, \qquad \text{so} \qquad \lceil \log_2 |\Omega| \rceil \mathbin{\dot{-}} \lceil \log_2 |\Omega'| \rceil \;\le\; \mu(s') - \mu(s).$$

Proof. Count the bill first. There are at least as many paid steps as levels in the tree. Each paid step adds at least one and every other charge is nonnegative. The total bill is therefore at least the tree's depth. Conservation says that bill is exactly the counter's rise, $\mu(s') - \mu(s)$.

Now count the room the tree supplies. It has at most two to its depth leaves. The covering premise allows at most $|\Omega'|$ prior entries per leaf, so

$$|\Omega| \le L(T)|\Omega'| \le 2^{\mathrm{depth}(T)}|\Omega'| \le 2^{\mu(s')-\mu(s)}|\Omega'|.$$

That is the first bound.

For the bit version, let $k = \lceil \log_2 |\Omega'| \rceil$. The posterior has at most 2^k entries. Insert that count in the first bound: the prior has at most $2^{\mu(s')-\mu(s)+k}$ entries. So numbering its entries needs no more than the counter's rise plus k bits. Subtracting the posterior's k bits gives the claimed bound; if the subtraction would go negative, the stipulated subtraction stops at zero. An empty prior already has bit count zero and satisfies the same conclusion. ■

Two arguments are stuck together in that proof, and they come apart. The tree half (the covering and the lemma) bounds the index bits by the sum of the charges of the run, on any price list at all. Conservation is only what turns that sum into the counter's rise. So the form in terms of cost, with the sum of the charges in place of the counter, holds in any Thiele machine, and it's part (3) of Theorem 10.1 below. The shape of the bound shows in what's missing from the hypotheses: anything tying the lists to the run.

There are three honest pieces of small print, all of them visible in the statement. Both logarithms round up separately, and the subtraction stops at zero, so when the prior and the posterior round up to the same number of index bits (four candidates narrowed to three, say) the rounded bound says nothing, though the first form of the theorem charges the run one unit: one leaf can't cover four candidates with three. The lists are plain lists, repeats allowed, so “index bits” is a difference of rounded list lengths with repeats counted; a list can get shorter by dropping a duplicate without losing a single candidate, and “a candidate was ruled out” and “the list got shorter”

are two different claims. And the difference isn't the number of questions anyone asked.

The covering condition, $|\Omega| \leq L(T) \cdot |\Omega'|$, is the nearest thing the count has to an anti-stopped-clock clause, and I have to be exact about how near. Picture each survivor standing for a little crowd of the prior's candidates, a crowd no bigger than the tree has leaves. The condition says the crowds can hold the whole prior between them: share the prior's candidates out among them and none overflows. That limits how much can be deleted per leaf you paid for, but it doesn't say why anything was deleted. So the condition rules out deleting too much for the tree you've paid for, and it doesn't rule out deleting for no reason. A run can satisfy every hypothesis of the count and have ruled nothing out on purpose; the count prices the size of the drop and says nothing about the honesty of it. And the covering can't just be dropped. Take sixteen candidates, the numbers 0 through 15, and a run of three paid steps whose one test asks "is it at least 15?" One yes-or-no question with a big constant in it narrows sixteen candidates to one, four index bits, on a ledger of three. A tree no deeper than three has at most eight leaves, and eight times one survivor is less than sixteen, so the covering fails, and without it the bound would say $4 \leq 3$. More candidates only widen the gap: 2^n candidates and the constant $2^n - 1$ give n index bits for the same cost of three.

A side trip is worth taking here, because it looks like it breaks the count and doesn't. Two information theorists, Rudolf Ahlswede and Gunter Dueck (1989), noticed that asking "is it this one?" is a different job from asking "which one is it?" If each receiver only wants to know whether the message is theirs, a noisy channel used n times can serve a number of receivers that grows like two raised to two to the nC, with C the channel's capacity, where naming the message only gets you two to the nC. The task is called identification, and on its face it says checking can be exponentially cheaper than naming. But their tests are allowed to say yes wrongly once in a while. Make the yes sound, a test that never says yes to the wrong message, which is the only kind of check this book lets raise a record, and the double exponential collapses. The argument is two lines of pigeonhole. Each receiver's test, to be any use, says yes on some output its own message can produce, and soundness means no other message can produce that output; so those outputs are all different, one per receiver, and there can't be more receivers than outputs, $|Y|^n$ for an output alphabet Y. That bound is single exponential, the same order as naming. So for a check that never lies, the card count is the right scale after all, and the cheap trick needs a check that can be fooled.

4.2 When the run does the narrowing

Theorem 10.1 takes the lists in. In the version below, the run makes the posterior, and the count stops being an item on the list and becomes a consequence. That version needs one thing a bare Thiele machine doesn't have: which steps are tests, and what each asks. So a reading names them. For each instruction it gives either nothing or a yes-or-no question about the state the run is in when it gets there, and a test step *passes* when its question says yes there. The candidates are states the run could have started from. A passed test is asked of a candidate by running the same moves from the candidate up to that step and asking the same question there. The *posterior* is the candidates that pass every test the run passed, asked that way, in the order the run asked them.

□ **Theorem 4.3** (Structural entitlement, with the narrowing made by the run)**.** Take a Thiele machine and a reading that names its test steps, every test step charged at least one. Take a state s that reads no, a run t from s that ends in a state that reads yes, and a list Ω of candidate starts that contains s. Let Ω' be the posterior t makes from Ω, and let b_i and a_i be the numbers of candidates standing just before and just after the ith test t passed. Then (1) Ω' sits inside Ω and contains s; (2) every candidate of Ω that isn't in Ω' fails, asked that way, a test t passed; (3) $|\Omega| \cdot \prod_i a_i = |\Omega'| \cdot \prod_i b_i$; (4) if every test t passed keeps at least a 2^{-k} share of the candidates standing before it, $b_i \leq 2^k a_i$, then $\lceil \log_2 |\Omega| \rceil \mathbin{\dot{-}} \lceil \log_2 |\Omega'| \rceil$ is at most k times the number of tests t passed, that number is at most the number of steps charged at least one, and the index bits are at most k times the sum of the charges; and (5) some step of t took the reading from no to yes and was charged at least one.

Proof. (1) Start with the prior list. Each passed test only removes candidates; it never inserts one. The actual start s passes every test used to filter the list, because these are precisely the tests the actual run passed. So s remains throughout, and the final list is a nonempty part of the original.

(2) Follow any candidate that is absent at the end. It was present at first, so there is a first test at which it disappears. The filter removes it only when its version of that test says no while the actual run's says yes. That test is the required witness.

(3) Write the list lengths on the bench: q_0 before any passed test, q_1 after the first, through q_n after the last. Here $q_0 = |\Omega|$ and $q_n = |\Omega'|$. Every before-count b_i is q_{i-1} and every after-count a_i is q_i. The left product in the statement is therefore $q_0 q_1 \cdots q_n$; the right product has the same factors with q_n placed first. They are equal, including when there are no tests and both products are empty products, equal to one.

(4) At the first test the share premise gives $q_0 \le 2^k q_1$. At the second it gives $q_1 \le 2^k q_2$. Substitute one into the other, and continue. After n tests, $q_0 \le 2^{kn} q_n$. If j bits number the final candidates, then $q_n \le 2^j$, so $kn + j$ bits suffice to number the original ones. The rounded bit drop is at most kn. Each of the n passed tests is a separate paid step, and each adds at least one to the nonnegative bill. Thus n is at most the number of paid steps and at most the total charge, giving the two cost comparisons.

(5) Read the actual run from its initial no until its first yes. The preceding reading was no, so that step raised it. The toll charges that step at least one. ∎

Part (3) is the exact count: take base-two logarithms and $\log_2 |\Omega| - \log_2 |\Omega'|$ is the sum, over the tests the run passed, of $\log_2(b_i/a_i)$, the bits each one took. Part (2) answers the complaint I made about the covering, that it doesn't say why anything was deleted: here everything deleted was deleted by a test the run passed and paid for. Two premises stay, the prior list and the share, a bound k fixed before the run on what one test can take. Every run meets some bound, its own biggest test's, so what the share adds is fixing it in advance, and that is needed. The one test "is it at least 15?" asked of sixteen candidates keeps one, a 2^{-4} share, and narrows by 4 index bits on a run of 3 paid steps; with $k = 1$ part (4) would say $4 \le 3$.

CHAPTER 5

From “Records over any base: the latch”

5.1 Bases from outside, and a relation I have to be honest about

The theorem is stated for any base, so let me put it on three models of computation that other people built: a small Turing machine (any program); a register machine of the kind the computer scientists Stephen Cook (of the certificates) and Robert Reckhow described in 1973, with unit-cost indirect addressing (any program); and a lambda calculus (call-by-value, meaning an argument is worked out before it’s plugged in, and made deterministic by letting a term with no step left to take step to itself). On each, the latch form holds, by the one generic theorem applied to that base (the bases are defined below). So a register machine with a base-driven record is a register machine plus a cup, and so is a lambda calculus.

I’ll give the three bases explicitly. A base needs a state, a next-state function, a set of starting states and a halting predicate; the latch theorem asks for nothing else about its instruction vocabulary.

Definition 5.1 (Three concrete bases)**.** The **tape base** has a finite Boolean list t, a head index h, a program counter q and a natural-number cost field. Reading beyond the list returns false. Writing at h extends the list with false cells if needed, then changes that cell. Moving right increments h and extends the list through the new head position. Moving left decrements a positive h; at $h = 0$, it prepends a false cell and keeps the head at zero. A fixed finite program contains these instructions:

Instruction	**Next state**
Write a bit	Perform the write and advance q by one.
Move left or right	Perform the move and advance q by one.
Branch to j	Set $q := j$ if the current cell is true; otherwise advance q by one.
Halt	Leave the entire state unchanged.

Instruction	Next state
ClaimTapeIsZero with a label ℓ	Advance q by one and do nothing else.

An out-of-range program fetch is Halt. Every state is allowed as a starting state, and halting means the fetched instruction is Halt. The cost field is preserved by every instruction in this base. In particular, its ClaimTapeIsZero instruction neither erases the tape nor charges the cost field; neither the name nor the label supplies any semantics.

The **random-access base** has a register function $X : \mathbb{N} \to \mathbb{N}$ and a program counter q. For a fixed finite program its instructions are:

Instruction	Next state
Constant (i, c)	Set $X(i) := c$.
Add (i, j, k)	Set $X(i) := X(j) + X(k)$.
Subtract (i, j, k)	Set $X(i) := \max(0, X(j) - X(k))$.
Indirect load (i, j)	Set $X(i) := X(X(j))$.
Indirect store (i, j)	Set $X(X(i)) := X(j)$.
Positive branch (j, a)	Set $q := a$ if $X(j) > 0$; otherwise advance q by one.
Jump to a	Set $q := a$.
Halt	Leave the entire state unchanged.

Each register-writing instruction advances q by one; its operands are evaluated in the old state, and all other registers are preserved. A fetch outside the program also leaves the state unchanged. Starting states have $q = 0$ and arbitrary register contents. A state is halted when it fetches Halt or is outside the program. Unit cost here counts instructions. It has nothing to do with the record's ledger.

For the indirect instructions, draw boxes 1, 2 and 3 and put 2, 7 and 0 in them. Indirect load $(3, 1)$ first reads the 2 in box 1, uses it as an address, and copies box 2's 7 into box 3. Now put 9 in box 3 and execute indirect store $(1, 3)$: the 2 in box 1 tells it where to put the 9, so box 2 changes and box 1 stays 2. An address is being read from a box; it is not necessarily the box's own number.

The **lambda base** uses the terms and weak call-by-value rule defined in Definition 7.1. To compute its next state, first try to reduce the function part of an application. If that part cannot step and is an abstraction, either substitute an argument that is already an abstraction or take a step in the argument. If neither applies, the term cannot step. Variables and abstractions cannot step by themselves, and there is no reduction under an abstraction. The base leaves a term unchanged

when it cannot step and calls precisely those terms halted. Every term is allowed as a starting state, including open terms.

So all three are bases in the sense of Definition 12.1, and the latch theorem applies to each. It says nothing else about them, nothing about what they can compute or how fast.

There's also a relation defined between bases, *weak base equivalence*: two bases are equivalent when some correspondence covers their start states both ways, related states agree on an observation and on halting, and one step on one side is matched by some number of steps, possibly zero, on the other. That "some number of steps" is the *stuttering* idea Leslie Lamport (the same Lamport as Section 8's message counters) put forward in 1983: a fine-grained machine and a coarse one shouldn't count as different because one takes three ticks to do what the other does in one. The relation carries no cost; a machine that multiplies huge numbers in one step and a Turing machine differ in price. The relation never looks at price, so on what it does look at they can match. It's proved to be an equivalence relation, and the latch form is preserved across it.

Draw a coarse machine $0 \to 1$, with 1 halted, and a finer one $a \to b \to c$, with c halted. Let a and b both show 0 and c show 1; halted states stay where they are. Relate 0 to both a and b, and 1 to c. The fine move $a \to b$ changes nothing visible, so the coarse machine matches it by taking no step. Its move $b \to c$ matches $0 \to 1$. Starting from a, the coarse move is matched by two fine steps. "Possibly zero" allows this; the relation is about matching observations and halting, not matching the number of ticks or their cost.

There is a limit to that statement. The "preserved across it" proof throws away its equivalence hypothesis; the latch holds on every base regardless, so the preservation is true for a boring reason. And nowhere in this book are *two* of the bases actually related by this relation. The relation is defined, it's an equivalence, and it is not used to connect the Turing machine to the register machine to the lambda calculus. It would be easy to read that list as if it did. The honest sentence is the one that's proved: each base, separately, carries a latch.

Definition 5.2 (Weak base equivalence)**.** Bases B_1 and B_2, with observations o_1 and o_2 into one common set, are *weakly equivalent* when some relation R relates every starting state of each to some starting state of the other, and any related x, y have $o_1(x) = o_2(y)$, the same halting, some $n \geq 0$ with $R(\text{next}_{B_1}(x), \text{next}^n_{B_2}(y))$, and some $n \geq 0$ with $R(\text{next}^n_{B_1}(x), \text{next}_{B_2}(y))$.

Theorem 5.1 (Outside bases, and the relation)**.** (1) Weak base equivalence is reflexive, symmetric and transitive. (2) If two bases are weakly equivalent, Observation 12.1 holds over one exactly when it holds over the other. (3) Observation 12.1 holds over each of the three bases in Definition 5.1, for every fixed program where the base has one. □

Proof. (1) A base matches itself by pairing each state with itself and matching one step with one step. Reverse any matching between two bases and it gives a matching in the other direction.

For three bases, pair a state of the first with a state of the third whenever some middle state matches both. A start on either outside has a middle start beside it, and that middle start has a start on the far side. Observations and halting agree across the first pair and then across the second, so they agree across the outside pair too.

Follow one step on the first base. Its match supplies a finite sequence of middle steps. Match the first of those steps on the third base, then the second from where that match ended, and continue. Joining the finite sequences gives a match for the original step. If the middle sequence is empty, take no step on the third base. Do the same in the other direction for a step on the third base. This checks both stepping requirements and proves transitivity.

(2) The latch argument uses only a base step and the stated record rules. It therefore holds over both bases, whether or not they are matched. (3) Each of the three displayed constructions supplies that step, its starts and its halting reading. The same latch argument applies separately to each. ■

CHAPTER 6

From “The shadow, exactly”

6.1 How much gets destroyed, on average

The halving price is a worst case, because it prices the largest landing pile a move can have. The next question is the average one: if I don’t know which state the machine was in, cups and all, but I have some spread of chances over the states, how much do I lose, on average, about which state it was, once I know where it landed? Shannon’s conditional entropy measures exactly that (Section 3 built the entropy from scratch, so I won’t do it again here), and I’ll write it in bits.

□ **Theorem 6.1** (The halving price in bits bounds the average loss)**.** Take any spread of non-negative weights over the states, uniform or not, with finite total weight, on a machine of any size. (1) If every landing pile has at most N states, with $N \geq 1$, the weighted conditional entropy of the state given its landing spot is at most the total weight times $\log_2 N$ bits. (2) So on a machine with the halving price, that weighted information loss is at most its cost in bits times the total weight, even when the spread’s own entropy is infinite. For a probability distribution, the total weight is one and the average loss is at most the cost. (3) If a landing pile has 2^c states, the uniform probability distribution on it loses exactly c bits and attains that bound.

Proof. Look at one landing spot. Suppose the sources that land there carry total weight $w > 0$. Divide each source’s weight by w. Those shares now add to one: they describe the uncertainty about the source once you know this landing spot. There are at most N possible sources. The logarithm argument in Section 3 says the uncertainty is greatest when those possibilities have equal shares, and is then at most $\log_2 N$ bits. Multiply by w to put this pile back into the whole spread. Its contribution is at most $w \log_2 N$. A pile with zero weight contributes nothing.

Add the contributions. The pile weights add to the original total weight, so the same $\log_2 N$ multiplies that total. Under the halving price, $N = 2^{\text{cost}}$, whose base-two logarithm is exactly the cost.

Infinitely many piles cause no subtraction of one infinite entropy from another. I am adding the nonnegative uncertainties inside the piles. Every finite collection contributes at most its weight times the bound, and its weight is at most the total. Taking the largest value approached by those finite sums keeps the same bound.

Finally put all probability on one pile of 2^c sources, equally shared. Before landing there are 2^c equally possible origins; afterwards the landing spot distinguishes none of them. The uncertainty left about the origin is $\log_2(2^c) = c$ bits. That attains the bound. ∎

Where does it stop? My first guess was one place, a landing pile with infinitely many states, and that the average would blow up there right along with the worst case. That's half right, and the half that's wrong surprised me. No finite charge satisfying the halving price can price a move that sends infinitely many states to one spot. The weaker merge price could still charge one, since it does not measure the pile's size. But the average depends on the spread. Put weight $\frac{1}{2}$ on the first state, $\frac{1}{4}$ on the next, $\frac{1}{8}$ on the next, and so on, and the information lost by crushing the pile never reaches 2 bits, however many of its states you count. Under that spread, a two-bit information budget covers the average, though no finite budget covers the worst case. So I went looking for a spread that does blow up, and it took a nastier one: blocks of states, block k holding 2^{2^k} states and a share $\frac{1}{(k+1)(k+2)}$ of the weight, which adds up to one. Under that spread the loss grows past every bound, and no finite information budget covers that average loss.

You can generate the first spread by tossing a fair coin until the first head. Zero tails names state 0, one tail names state 1, and so on. State j has chance $2^{-(j+1)}$, and identifying that outcome takes $j+1$ toss-results, so its entropy term is $(j+1)/2^{j+1}$. Add these by counting each possible toss position: the first is reached with chance 1, the second with chance $1/2$, the third with chance $1/4$, and so on. The total is $1 + 1/2 + 1/4 + \cdots = 2$; every finite partial sum of the state terms is smaller. Resetting all those outcomes to one box can lose only those two bits on average, despite having infinitely many possible starts.

For the block spread, split each block's weight equally among its states. The block weights sum to one because $1/((k+1)(k+2)) = 1/(k+1) - 1/(k+2)$. Within block k, there are 2^{2^k} equally likely choices, so just identifying a state within that block contributes $2^k/((k+1)(k+2))$ to the average bits. Those contributions grow without bound as k grows, and forgetting which block it was only adds to the loss. And no finite worst-case price can handle an infinite pile: for any proposed cost c, select $2^c + 1$ different states in it. They all land together, violating the halving rule's maximum of 2^c.

□ **Theorem 6.2** (An infinite pile)**.** (1) A move that sends infinitely many states to one spot isn't priced by the halving price at any finite cost. (2) Under the halving spread $2^{-(j+1)}$, crushing the pile loses exactly 2 bits, and counted over its first n states, less than 2 for every n. (3) Under the block spread, the information lost grows past every bound, so no finite bit budget covers it.

■ *Proof.* The three computations above.

And heat? Part I already said where the line is, and it's the same line here. Landauer's principle, that crushing c bits' worth of possibilities gives off at least $c\,kT\ln 2$ of heat on average, is a premise from physics, and a contested one (Section 3). Nothing in this section proves anything about heat. What the counting does give is that, under that premise, on a machine with the halving price, a move's cost priced at $kT\ln 2$ per unit, times the weight of the piles, is always enough to cover the minimum the premise would demand, for every spread of finite total weight. If the premise falls, that sentence falls with it, and every theorem above it stands, because none of them used it.

CHAPTER 7

From “The diagonal”

Two forms of the wall live in this section, and they’re honest in different ways.

The first is the *nat family*, a tiny substrate built so that program two is the Contrarian for a given candidate decider d, and for every d, d is wrong about program two. It’s a *family*, one instance per d, and its recursion theorem holds by construction. It’s the index cards made formal and nothing more.

Theorem 7.1 (The nat family)**.** Fix any Boolean function d on $\mathbb{N}$. Let the substrate N_d have programs and states $\mathbb{N}$; program 0 returns 1, program 1 returns 0, program 2 returns 0 if $d(2)$ is true and 1 if it is false, whatever the input state, and every other program gives nothing; each program is encoded as itself; μ is constantly 0, so any step relation will do; $\mathrm{Rep}(f)$ holds when $f(p) = 1$ for every p with $d(p)$ true and $f(p) = 0$ for every other p; and $A(p)$ holds when program p returns 1 on input 0. Then it is not the case that $d(p) = \text{true} \iff A(p)$ for every p. □

Proof. The recursion theorem of N_d holds with the fixed witness 2: for a representable f, $f(2)$ is 1 or 0 according to $d(2)$, and program 2 returns exactly what that program returns. The witnesses for A are 0 and 1, and A is extensional because it reads only what a program returns. The flip of d is representable by the definition of Rep, so Theorem 14.1 applies to d itself. Directly: $A(2)$ holds exactly when $d(2)$ is false. ■

Now push on it once more, because the nat family shows the outside decider in three lines, and you can do them yourself. Stand outside N_d, knowing d. Write down a new decider e: say yes on program 0, say “not $d(2)$” on program 2, and say no on everything else. Check it against A. Program 0 returns 1, so $A(0)$ holds and e says yes. Program 1 returns 0, so $A(1)$ fails and e says no. Program 2 returns 1 exactly when $d(2)$ is false, and that’s what e says. Every other program gives nothing, so A fails and e says no. So e is right about every program. Does that break the wall? It doesn’t, and the reason is the part that matters: in N_d a flip is representable only if it says 1 exactly where d says yes, so the flip of e is representable only if e and d agree

everywhere, and they disagree at program 2. The correct decider exists; its flip just isn’t one of the substrate’s own moves. (It’s three lines and a pencil, and you just did it.) One and the same property is undecidable from inside and decided cleanly from outside, at the same time. That’s not a paradox, because it’s two different people asking.

The second form is the one where the recursion theorem is *proved* from the reduction rules rather than assumed. It’s on *L*, the weak call-by-value lambda calculus that Yannick Forster and Gert Smolka presented in 2017 as a model of computation, and which a group at Saarland has since used as the seed of a whole library of undecidable problems. Here L is built from its reduction rules. The recursion theorem is proved, Rice’s theorem follows, halting follows, and the diagonal is reproved in the shape of the substrate theorem, for any property of programs that depends only on what they compute and that holds of one program and fails of another. The statement mentions no shortcut. So on L the result is the diagonal for one fixed model, and I don’t plug L into the substrate theorem, for this reason: a substrate’s run map has to answer, with a state or with nothing, and for L that map is the halting function itself, which no term of L computes. Using it would take something from outside L, and L is in this book as the model where nothing is taken from outside. So on L I run the argument again from scratch instead of applying the general one. Both are true; they’re different sentences.

I’ll say what that string of words means, because L is the strangest thing in this book that computes, and the smallest. The lambda calculus is Alonzo Church’s, from the 1930s, and it’s a programming language with one idea in it: functions. A term is one of three things: a variable; a function, written λ followed by its body, the body saying what to do with whatever you hand it; or one term applied to another, written side by side. Running a program is one move, over and over: when a function meets an argument, put the argument into the body where the variable was. The lambda calculus has no numbers, no memory, no instructions, and in 1937 Turing published a proof that it computes exactly what his machines compute. “Weak call-by-value” is two house rules about the order of moves: an argument is run down to a finished function (a *value*) before it’s put in, and nothing runs inside a function that hasn’t been handed anything yet. Variables are written as numbers counting how many λs back they were bound (a trick of Nicolaas de Bruijn; it means two variables can never get mixed up by sharing a name), and a term is *closed* when every variable in it is bound inside it, so it leans on nothing outside. Last, to hand a program to a program, L needs a way to write any term as data, the way a program’s source is a string you can pass around. The one used here is the *Scott encoding*, named for Dana Scott, a student of Church’s who went on to build mathematical models of the lambda calculus itself. As far as I can trace it, the encoding first appeared in lecture notes of his from Berkeley in 1962 that he never published, as a way to write data;

Torben Mogensen carried it over to lambda terms themselves in 1992, and that's the form used here. All you need about it is that every term t gets one *code*, written $\ulcorner t \urcorner$, and the code is itself a closed value. Two tools get built on top of that in the proof: a *quoting* term Q that takes the code of a term to the code of that code (a program that prints its own source needs exactly this move), and a *fixed-point combinator*, a term that lets a function call itself, which is how Q works its way through a code of any size.

Do two of these on paper before I give you the general rule. Write $I = \lambda x.x$: its instruction is "return whatever you were handed." Hand I its own card and you get $I\,I \to I$, and it stops. Now write $D = \lambda x.x\,x$: "make two copies of what you were handed and apply the first to the second." Hand it its own card. Putting D in for both copies of x gives $D\,D \to D\,D$. You're back at exactly the same instruction and argument, so the rule never finishes. Both arguments were already functions, so neither example sneaks any work under a function that hasn't been handed anything.

The variable numbers are what keep the copying from getting confused. In $\lambda x.\lambda y.x$, the last x points past the nearest λ (the one for y) to the one before it, so its number is 1; in $\lambda x.\lambda y.y$ the last variable has number 0. Rename x and y however you like and neither recipe changes. The codes below are a way of putting a whole recipe on a card that another recipe can read.

Definition 7.1 (L, as built here)**.** A term of L is a variable (a de Bruijn number), an application $s\,t$, or an abstraction λs; the values are the abstractions. One step is weak call-by-value, left to right: $(\lambda b)\,v$ steps to b with v substituted when v is a value, a step inside the left part of an application is a step of the application, and so is a step inside the right part once the left part is a value. A term s *evaluates to* v when s reaches v in some number of steps and v is a value; $p \equiv q$ when p and q evaluate to the same values. Every term t has a code $\ulcorner t \urcorner$, its Scott encoding, which is a closed value. A map f on terms is *L-computable* when some closed value s has $s\ulcorner p \urcorner \equiv f(p)$ for every p. A term D *decides* a property P of terms when, for every p, $D\ulcorner p \urcorner$ reaches $\lambda\lambda.1$ (true) if $P(p)$ holds and $\lambda\lambda.0$ (false) if it fails. P is *extensional* when $p \equiv q$ and $P(p)$ give $P(q)$.

Theorem 7.2 (Recursion, Rice and halting for L)**.** (1) For every closed value s there is a closed term t that reaches $s\ulcorner t \urcorner$; so every L-computable f has a term p with $p \equiv f(p)$. (2) Let P be extensional, with $P(y)$ and not $P(n)$ for closed terms y and n. Then no closed term decides P. (3) No closed term decides halting, the property of evaluating to some value. □

Proof. (1) First build a program that can assemble the code of the very application I am about to run. The quoting program Q takes a term's code and returns the code of that code. Appendix 1 gives the idea of its construction. The program mkapp takes two codes and joins them into the code of an application.

Make A do this when handed a code x: quote x, put that quoted code beside x using mkapp, and hand the result to s. Written out,

$$A = \lambda x.\, s\,(\text{mkapp}\ x\ (Q\,x)).$$

Now run A on its own code, and call that whole application t: $t = A\ulcorner A\urcorner$. The first step puts $\ulcorner A\urcorner$ where x stood. Quoting it supplies the code of $\ulcorner A\urcorner$; joining the two codes supplies the code of $A\ulcorner A\urcorner$. That is the code of t itself. The computation therefore reaches $s\ulcorner t\urcorner$, as promised. All the pieces were closed, so t is closed too. If s computes f, the resulting behaviour is exactly $f(t)$.

(2) Suppose a closed program D decides P. Build another program that asks D about the code it receives, then runs the no-example n after yes and the yes-example y after no. In this language you must keep the two branches wrapped as functions until the answer chooses one; otherwise call-by-value would try to run them before choosing. The following term does that:

$$\lambda x.\, D\,x\,(\lambda_.\, n)\,(\lambda_.\, y)\,(\lambda z.z).$$

The final identity function is the harmless argument that opens the selected wrapper. Thus a true answer runs n, and a false answer runs y.

Apply part (1) to this program, obtaining its self-matching t. If $P(t)$ holds, the judge says true and t behaves like n, where P fails. If $P(t)$ fails, the judge says false and t behaves like y, where P holds. Either answer contradicts the requirement that alike behaviour has the same verdict. No such D exists.

(3) Halting depends only on which values a term reaches. The identity $\lambda z.z$ is already a value. The term $(\lambda x.\, x\,x)(\lambda x.\, x\,x)$ takes a step back to itself each time, and is never a value. These supply the yes-example and no-example for part (2), so halting has no closed decider either. ∎

The construction in that proof isn't mine, and the credit belongs right here. Forster and Smolka's L development of 2017 has the Scott encoding of terms, a closed quoting term with exactly this specification, the code of a term to the code of its code, and a fixed point built the same way: their t is $C\ulcorner C\urcorner$ with $C = \lambda x.\, s\,(A\,x\,(Q\,x))$, so their C is my A, and their A, which builds the code of an application, is my mkapp. They say they follow the proof in the Dutch logician Henk Barendregt's textbook on the lambda calculus (1984), where statement (1) is the full lambda calculus's second fixed-point theorem, and part (2) is what they call

Scott's theorem. The idea of the term construction is in Appendix 1; the route is theirs, and Barendregt's before them.

So I've given the wall on paper, on a family built to fit it, and on one outside model.

The table below lines up the picture all of that fills in against Turing's, one column per axis.

Time axis	**Structural axis**
counts: steps	counts: μ
asks: does it halt?	asks: does it admit the shortcut?
the wall: no Turing machine decides halting	the wall: no decider whose flip is representable, under a recursion theorem

Read the table as two uses of one diagonal, each under its own premises, and as saying nothing more than that. The structural wall never uses μ; the μ entry only says what that axis counts. The nat result is a family indexed by the candidate decider, and L's results are about one fixed model and properties that mention no shortcut. Neither hands you a second fixed undecidable problem, and neither separates Turing degrees: a property can sit behind the wall from inside and be decided cleanly from outside, as the pencil argument on N_d showed. And "admits the shortcut" in the right-hand column is the theorem's sense, any behavioural property with a yes and a no. "Has a cheaper way to the same answer" is a second question, and the column has no wall for it.

CHAPTER 8

From “What established means”

8.1 The guy who got closest

Somebody had that clue before me, and on that clue he’s the closest neighbour I’ve found, so he gets the most space and the hardest push.

Bennett again, the one from Part I’s counting of states and the demon, wrote a paper in 1988 that starts where I just ended up: information isn’t the same as value. His example is an almanac of where the moon and planets will be every day for a hundred years. It has no more information than the equations it was calculated from, but it saves you doing the calculating. So the value is the work. He called the amount of work a thing carries its *depth*.

Was this my idea, already done, by a guy at IBM? The only way to find out was to find where it parts from mine, because if I couldn’t find that place I’d just be renaming his idea. And it’s one word: *plausibly*. Depth is the amount of work a thing was *plausibly* made with. Bennett says straight out that you can’t know how a message actually came to be; for all you know it came from flipping coins. Depth is read off the finished thing, the way a fossil’s age is read off the fossil.

Bring the two computers back. The sorted list in the first one has some depth. The identical list in the second has exactly the same depth, because depth looks at the list and nothing else. A photocopy of a deep thing is just as deep. Depth tells you what work a plausible way of making the thing would take. It’s not a record that *this* machine did it, and it’s not a receipt for what doing it cost.

So Bennett’s standing right next to the line, on the other side of it, and as far as I can tell he got his own question right. He asked how much work a thing embodies. I’m asking who has the right to lean on it and what they paid to get there. His answer lives in the object. Mine, if I’ve got one, has to live in the machine.

CHAPTER 9

From “Five small models”

The fragment is a three-instruction memory-bounds language with the policy “every address is below the limit”: the checker accepts exactly when the condition holds, and a certificate implies the condition. One example sits on top of that, a computation you can do in your head: a program that reads address two under a limit of two is rejected. The fragment isn’t the paper’s real case study, and nothing connects it to anything else in this book. And it has a limit: a program that writes garbage to address zero passes if zero is below the limit. The proof is of the stated policy and nothing else.

And what was the paper’s real case study, the one my fragment isn’t? It was network packet filters, the little programs an operating system runs on incoming network traffic to decide which program gets to see which packet. They run inside the kernel, the most trusted part of the system, and they have to be fast, so they’re exactly the code you’d most like to take from a stranger and least like to trust. In the 1996 paper that first set out the idea, Necula and Peter Lee wrote several filters by hand in the assembly language of a DEC Alpha, shipped each with its proof, and reported a one-time cost of 1 to 3 milliseconds to check the proof, after which the filter ran with no checks at all. Those milliseconds put a number on “checking is cheap by design”. What the paper reports changes nothing about my fragment, which still isn’t that case study, and it doesn’t add a price: the milliseconds measure the checker’s work in that experiment. They are not a per-crossing toll theorem, whether or not an implementation saves the result.

Definition 9.1 (The bounds fragment)**.** Instructions are READ a, WRITE $a\ v$, and HALT. For a memory limit L, the policy allows READ a and WRITE $a\ v$ when $a < L$, and always allows HALT. The *verification condition* of a program is that the policy allows every instruction in it. A *certificate* is a list of per-instruction policy facts, one for each instruction in order. The *checker* tests the policy on each instruction and returns the conjunction.

□ **Theorem 9.1** (The checker is the verification condition)**.** For every limit L and program P: the checker accepts P exactly when the verification condition holds; any certificate for P gives the verification condition; and with $L = 2$ the checker rejects READ 2 followed by HALT.

Proof. Lay the instructions in a row. For each READ or WRITE, compare its address with L; put yes beneath it exactly when the address is smaller. Put yes beneath HALT without asking about an address. The checker accepts exactly when every mark in that row is yes. The verification condition asks for exactly those same per-instruction facts. An empty row has no failing instruction, and extending the row preserves acceptance precisely when the new instruction also passes. This covers programs of every finite length.

A certificate supplies one policy fact at each place in the same row. Reading them in order gives every fact the verification condition asks for, so it implies that condition too.

With limit two, the first instruction READ 2 asks whether $2 < 2$. It does not. The later HALT is allowed, but cannot undo that failed comparison, so the checker ■ rejects the two-instruction program.

9.1 One example, three levels

Before I lay all five side by side, let me walk one of them up and down the three levels from the start of the section, because every claim in here lives at exactly one level and it’s easy to let a claim slide up a floor without noticing. I’ll take gas, since it’s the one place all three line up. Think of a restaurant’s real recipe, my photocopy of one page of it, and a cookbook chapter on what any recipe has to contain. I can check my photocopy line by line. That tells you nothing about what the kitchen cooks tonight.

Start with the real specification. Ethereum’s execution specification says, in a document that runs real money, what a storage write costs, what clearing it refunds, and how the refund is capped at the end of the transaction. None of what I proved is at this level. What the network actually charges is whatever the clients people run actually compute, and the evidence for that is the clients and the chain. This book has none of it.

Next comes the transcription. I copied one mechanism out of that document, one slot in one transaction, gas and refund arithmetic only, at a named commit (Definition 16.2, Theorem 16.3). What the result tells you: *if* the storage rule is what I copied, then leaving a fresh slot set costs at least twenty thousand, and

set-then-clear nets twenty-three hundred at the counter. What it can't tell you: anything about the refund cap (not copied, which is why what a real set-then-clear pays came out so different from the counter earlier), the stack, calls, other slots, or whether any client implements the rule as written. The boundary of the claim is the edge of the copy.

Last comes the wrapper. The gas schedule of Definition 16.3 isn't Ethereum. It's any system with steps, a cost per step and a yes/no commitment flag, plus one thing *stipulated*: a charge predicate, with the premise that a charged step costs at least one and an uncharged one costs zero. That premise is the whole of what the wrapper adds, and it's added by hand; no specification supplies it. Given it, the theorem says every run costs exactly its number of commits, never letting a commit go unbilled, never billing one twice and never billing a non-commit, exactly when the charge predicate is the commit predicate at unit price. What that tells you is what exact pricing would have to look like, in any system. It doesn't say Ethereum meets it, and Ethereum doesn't: real gas bills computation that commits nothing, which the transcription shows in numbers. So the wrapper prices a definition, the transcription prices a copied rule, and neither prices the chain.

The other four sit on the same three levels, and only one more wrapper is spelled out in this section. Casper's transcription is the accountable-safety proof above; its wrapper is a small gadget with the stake at risk handed in. A finalize step that risks nothing turns the record on at no charge, which breaks Section 2's toll, and one that risks at least one unit has a floor. It records exposure to loss (no collateral is destroyed in it), has no Casper checkpoints in it at all, and is priced by stipulation, the same way the gas wrapper is. Attestation, transparency and proof-carrying code have transcriptions that prove what their verifiers do and no price theorem, because no specification of theirs puts a price on the verdict. Their wrappers, which I don't spell out here, are Section 9's verifier argument told in each system's vocabulary, with a colliding pair behind each one, and that argument is all they carry here.

The book's rule is that every claim gets a way to break it, so I'll say how to knock down the four claims with the most weight on them. For the gas transcription: a list of stores from a fresh slot that ends nonzero with net charge under 22100. For the gas wrapper: a schedule in which every run costs exactly its number of commits, whose charge predicate still differs from the commit predicate at some step. For the finality gadget: a finalizing trace from unfinalized to finalized whose charged stake is zero while the gadget claims a floor. For the attestation fragment: a verifier reading only the quote's fields that decides a runtime flag the quote ignores.

CHAPTER 10

From “The pointer thesis”

10.1 What’s proved

This section has definitions, two toys, a theorem about independence, a survey, and two refutations of things that would have been stronger.

The definitions are the ones above, plus “unique pointer among rivals”: the event proliferates and none of a listed set of rival events does. Uniqueness is *relative to the list*, necessarily, since any event that holds on exactly the same states as a proliferating event proliferates whenever it does: the same fact under another name, so someone has to say which names count as rivals. And the first criterion is stated as a schema: for every ecosystem in some class, its metered event is the unique pointer among its rivals. The class and the rivals are inputs. Nothing about them is proved. A second set of definitions, after this one, tightens “proliferates” to “copied and relied on” and drops the rival list.

The formal object is small enough to hold in one hand.

Definition 10.1 (Ecosystem, records, proliferates, pointer)**.** An *ecosystem* is a type of states, a number N of observers, and for each observer i a Boolean fragment F_i on states. Observer i *records* an event E (a property of states) when, for every state s, $E(s)$ holds exactly when $F_i(s)$ is true. E *proliferates* when every observer $i < N$ records it. E is the *unique pointer among* a list of rival events when E proliferates and no rival in the list does. Given a class C of ecosystems, and for each member a designated metered event and a list of rivals, the *pointer criterion holds* for C when every member’s metered event is the unique pointer among its rivals.

Nothing in the definition makes two fragments read different parts of the state, and “metered” is only a label: the definition has no metering in it.

One bit per observer can’t even state the objection a replicated system raises. Every full node of a blockchain holds the whole shared state, so every fact about it, “gas used is at least one” as much as “final,” sits on every node. The tightened definitions give each observer whatever it holds, an action it takes on what it holds, and a flag for whether it produced the record.

Definition 10.2 (Copied and relied on, independent carriers, the pointer)**.** An *ecosystem with reliance* is a type of states, a number N of observers, and for each observer i a view V_i (a function from states to some type of things an observer can hold), an action a_i (a Boolean function of the view) and a producer flag. Fix a set R of reachable states. An event E is *copied* on R when for each $i < N$ some Boolean function d of the view has $E(s)$ exactly when $d(V_i(s))$ is true, for every s in R. Observer i *relies on* E when $a_i(V_i(s))$ true gives $E(s)$ for every s in R, and $a_i(V_i(s))$ is true for some s in R. E is *relied on by parties who didn't produce it* when some observer $i < N$ is not a producer and every such observer relies on E; it is *copied and relied on* when it is both. E is the *pointer* on R when it is copied and relied on, and every event that is copied and relied on holds at every state of R where E holds. The carriers are *independent* under a family of relations P_i (states that agree on observer i's own part of the world) when each view gives equal values on states related by its P_i, and for any two observers $i \neq j$ below N and any states s, t there is a state u with $P_i(u, s)$ and $P_j(u, t)$: the two parts can be set separately.

Two pointers on the same R agree on every state of R, so the pointer is unique with no list of rivals. And the definition of independence has a consequence I didn't put in on purpose.

Put one yes/no card in front of each of two readers. If the cards can be set separately, all four arrangements are allowed: no/no, no/yes, yes/no and yes/yes. Ask each reader to recover one shared event using only their own card. Suppose that event is false at no/no and true at yes/yes. At no/yes, the first reader sees exactly what they saw in the false case, while the second sees exactly what they saw in the true case. Their answers conflict. That mixed arrangement is exactly what the independence condition hands the proof.

Now impose a copying procedure that only leaves no/no or yes/yes. On those reachable arrangements each reader can report the shared bit correctly. Separate cards are things that *could* carry a copy; what makes them copies is that the arrangements you actually reach have the two agreeing. The theorem below is about recovering the event exactly on *all* arrangements, the mixed ones included, which the copying never leaves behind.

Theorem 10.1 (Independent copies on every state: the event is equivalent at any two states)**.** In any ecosystem with reliance whose carriers are independent and which has at least two observers, an event copied on the set of all states holds at one state exactly when it holds at any other, so it holds at every state or at none. □

Proof. Let observers 0 and 1 decide the event by d_0 and d_1. Given states s and t, take u with observer 0's part from s and observer 1's from t. Then the event at s is d_0 of observer 0's view, which is the same at u, so it equals the event at u; and the event at u equals the event at t the same way through observer 1. ■

So separate parts of the world can each carry an exact copy of a real record only on the states the world actually reaches, where the copying made them agree, and that's why the definition asks for copies on a set R of reachable states. Zurek's copies are the same: the light carries the dial's reading because the dial put it there.

□ **Theorem 10.2** (A replicated header: copying doesn't separate, reliance does)**.** Let the states be a block header (a finalized bit, a gas-used count) and three nodes' copies of it, with R the states where every copy equals the header; node i's view is its own copy, its part of the world is that copy, node 0 is the producer, and every node acts exactly when its copy says finalized. Then the carriers are independent; “finalized” and “gas used is at least one” are both copied on R; “finalized” is the pointer on R; and “gas used is at least one” is not relied on by parties who didn't produce it.

Proof. Independence: to give node i the copy from s and node j the copy from t, overwrite node j's copy in s. On R every copy is the header, so each node reads either fact off its copy. Node 1 acts only where its copy, and so the header, says finalized, and acts at the finalized header with no gas used; that state shows gas used fails node 1's reliance. Any event relied on by node 1 holds wherever node 1 acts, which on R is wherever the header is finalized, so “finalized” is the strongest. ■

It's a sanity check with maps I wrote, and it shows only that the tightened definition can tell the two apart where plain copying can't. Which of the two it picks, the actions pick: let every node act when its copy says gas was used, and gas used is the pointer. When what the other parties act on is copied, that is the pointer, so here the argument lives in the actions, and I wrote those too.

The toy has three observers on a replicated ledger, where certification proliferates and “work done” doesn't. It's an illustration, and that's all it is; it isn't evidence.

□ **Theorem 10.3** (The toy)**.** Let the toy have states (a certified bit, a work count), three observers, and every fragment equal to the certified bit. Then “certified” is the unique pointer among the one rival “work at least 1”.

Proof. Every fragment is the certified bit, so every observer records "certified" by definition. For the rival, take the state with the bit false and work 1: the event holds and observer 0's fragment is false, so observer 0 does not record it. ∎

The survey covers twelve events in one theorem: for each of the five systems in Section 16, the designated event (finalized, committed, attested, logged, checked) proliferates in the observer map I wrote for that system, and a rival (proposer work, a scratch counter, measurement noise, submission latency, prover steps) doesn't. The five together are one statement, an instance of the schema above. The survey adds a symmetric message-authentication code, which does *not* proliferate, and a digital signature, which does, for twelve rows in all. The five maps all use the same construction: a state has a designated Boolean event and a separate nonnegative rival count, three observers each return the event bit, and the rival asks whether its count is at least one. The bit and count can vary independently. The signature map likewise gives all three observers the authentic-signature bit. In the MAC map, observers 0 and 1 receive the authentic-tag bit and observer 2 always returns false. These maps, not a protocol running on a network, are the entire tests here. And there's a thirteenth, which I think is the most important one: swap which bit every observer copies, and the criterion picks the *other* event. The criterion selects whatever the observer map copies. The map is where the argument lives, and I wrote the maps. So the five "confirmations" are five instances of one construction with the fields renamed. I'm saying that twice on purpose, so it can't slip past once.

Theorem 10.4 (The survey, and the swap)**.** (1) In each of the five observer models (proof-of-stake, gas, attestation, transparency, proof-carrying), the designated event proliferates and the designated rival does not; in the symmetric-MAC model the authentic-tag event does not proliferate; in the signature model the authentic-signature event does. (2) Let the states be pairs of bits (first, second), with three observers whose fragments all equal the second bit. Then "second is true" is the unique pointer among the one rival "first is true". □

Proof. Each confirmation holds by definition: each observer's fragment is true exactly when the event holds. Each refutation exhibits one state where the event holds and some observer's bit is down. In (2) that state is (true, false). ∎

Two stronger claims are both refuted, and I'd ask you not to run them together, because they fail for different reasons.

The first stronger claim is that *every forgery-resistant commitment proliferates*. If that were true, certification would be the pointer because it's the commitment. It's refuted in three observer models (deniable authentication, where only the

designated verifier’s bit tracks the event; a symmetric MAC, where the key holders share a secret and nobody else can check; an object capability, an unforgeable token that grants access to whoever holds it) and confirmed in two (a public log; a digital signature). Build the first with three observers: give one a copy of the event’s bit and the other two a constant false. The one records it; it doesn’t proliferate. The proved half is the map computation. Whether my maps are *fair* to the real designs is argued in prose, and you should argue back.

□ **Theorem 10.5** (One blind observer is enough)**.** (1) In any ecosystem, if some observer $i < N$ has $F_i(s_0)$ false at a state s_0 where E holds, then E does not proliferate. (2) Let the deniable model have states (authentic, verifier-accepts), three observers, observer 0’s fragment the authentic bit and observers 1 and 2 constantly false. Then observer 0 records “authentic”, and “authentic” does not proliferate. The symmetric-MAC and object-capability models also fail to proliferate their events; the public-log and signature models proliferate theirs.

Proof. (1) Recording would require $E(s_0) \iff F_i(s_0)$, and the left side is true while the right is false. (2) Observer 0’s fragment is true exactly when the event holds. Apply (1) with observer 1 and the state (true, true). The other two refutations apply (1) to a blind observer of their own model (the symmetric-MAC map gives observer 2 a constant false; the object-capability map gives the token holder the bit and the other two constant false). The two confirmations hold by definition: the public-log map gives all three observers the logged bit, and the signature map gives all three the authentic-signature bit. ■

The theorem is about the observer maps, and only them. That a third party in deniable authentication really cannot be convinced is the modelling claim, and the model has no transcript, key or verifier in it to check that against.

The second stronger claim is that *agreement plus accuracy plus locality imply permanence*. That is, if all observers agree, each sees the event exactly, and each updates by the same local rule with no coordinator, then the recorded event can’t be undone. The smallest thing imaginable refutes it: one bit, two observers who both see it, and the rule “flip.” All three conditions hold and true becomes false in one step. Do it on paper; it takes a minute and it’s humbling. What *does* give permanence is durability, “an observer’s true view stays true,” added as a premise, and then it follows in a few lines. So durability is the premise doing the work, and durability is Section 3’s permanence wearing a different coat. Agreement doesn’t buy it. Do the paper version again with one change, the rule “stay true once true” in place of “flip,” and the bit can’t fall, because the rule won’t let it. The observers agreeing has nothing to do with it. (The latch of Section 12 has a reading that’s

durable in exactly this sense: once it's up, its step rule never brings it down.) None of this rules out a narrower theorem with real assumptions about the network, the adversary or durability; it rules out these two strong forms.

Definition 10.3 (Observation game)**.** A *game* has states, a number N of observers, a Boolean event e, a Boolean view V_i for each observer, and a step map. For $i, j < N$ it has *consensus* when $V_i = V_j$ on every state; *authenticity* when $V_i(s) = e(s)$ for every state and every $i < N$; *coordinator-free update* when one function ℓ on Booleans has $V_i(\text{step}(s)) = \ell(V_i(s))$ for every state and every $i < N$; *durable views* when $V_i(s)$ true gives $V_i(\text{step}(s))$ true for every $i < N$; and a *permanent event* when $e(s)$ true gives $e(\text{step}(s))$ true.

Theorem 10.6 (What buys permanence)**.** (1) It is false that every game with $N > 0$, consensus, authenticity and coordinator-free update has a permanent event. (2) Every game with $N > 0$, authenticity and durable views has a permanent event. □

Proof. (1) The toggle game: states are Booleans, two observers, event and both views the identity, step "flip". Consensus and authenticity hold trivially, the local rule is "flip", and the state true steps to false. (2) If $e(s)$ is true, observer 0's view is true by authenticity, stays true by durability, and authenticity turns it back into $e(\text{step}(s))$. ■

10.2 What isn't proved, and must stay that way on the page

What isn't proved is that certification *in deployed systems* is singled out by being copied and relied on across *independent* carriers. The criterion defines independence; it has no precise definition of "deployed" or "faithfully model," and no cost law connecting being copied and relied on to price. Every correspondence between a mathematical observer map and a real system is a sentence I wrote. The thesis is a thesis, and anyone who says the formal object proves it is reading more than is there.

10.3 What the thesis asks of the examples

There are three concrete comparisons to make. First, look at the record a deployed discipline prices. Do parties who did not produce it copy and rely on it? A priced record that nobody else relies on would contradict the thesis's second half.

Second, compare certification with rival events. If an event not implied by certification is copied and relied on to the same standard, certification is not the

unique pointer. Multiplication counts or head-turn counts would qualify if they met that same test in the system being described.

Third, apply the independence condition of Definition 10.2 to a faithful model of the ecosystem and ask whether certification itself meets the test. The symmetric-MAC example shows why the carriers and their access matter. The two stronger forms already refuted above help locate the distinction this thesis needs.

These comparisons supply the evidence the worldly claim asks for. The mathematical criterion supplies a definite question once the model and its observer maps are chosen.

CHAPTER 11

From “The counter against physics”

11.1 The calibration is a unit conversion

Multiply the counter by an energy per unit of cost that you choose, and you get a theorem: a state with one unit, at the scale kT ln 2 per unit, carries kT ln 2 of energy, for every k and T. Read that for exactly what it is: a unit conversion, true by definition, with the scale as an unforced input. Nothing in it picks the scale. The theorem would be just as true with the scale set to a dollar a unit. What *would* connect units of cost to joules is the premise in Section 3, Landauer’s principle, which is a physical premise. And even as a scale, $k_B T \ln 2$ a unit is the floor for one particular squeeze, a halving. Corollary 3.13’s floor for a flip that brings k states in beside m already reading yes, starting from the even spread on those $m + k$ states, is the larger of $k_B T \ln \frac{m+k}{m}$ and $\frac{2k}{m+k}\, k_B T \ln 2$, and for one newcomer among many yes-states that’s far below a halving, about $1.4\, k_B T / m$. Price every unit at $k_B T \ln 2$ and you can overstate the floor of a single narrow flip by a factor that grows with m.

Remember the constant from Boltzmann’s grave in Section 3, the exchange rate between a count and heat? It has a newer chapter, and it makes the point sharper than I can. The k_B in $k_B T \ln 2$ turns a temperature into an energy, and for most of its history it was a measured number, with error bars, pinned down by careful experiments. Since 20 May 2019 it isn’t. That day the international system of units was redefined so that Boltzmann’s constant is exactly 1.380649×10^{-23} joules per kelvin, by definition, and the kelvin is whatever makes that true. So the conversion from temperature to energy has been a convention since then, in the strictest sense: its numerical value in those units is fixed. Realising the kelvin in a laboratory still requires measurements, with uncertainty. What’s still physics is everything around it, whether wiping a bit at a temperature really sheds that much heat, which is what the Lyon bead was measuring. The calibration theorem is the same kind of move as that redefinition. It fixes a conversion, and the physics, if there is any, is somewhere else.

Definition 11.1 (Energy at a scale)**.** For a real number c (joules per unit) and a state s, the energy of s at scale c is $\mu(s) \cdot c$. The *Landauer calibration* at k_B and T is the statement that every state with $\mu(s) = 1$ has energy $k_B T \ln 2$ at scale $k_B T \ln 2$.

□ **Theorem 11.1** (The calibration is a unit conversion)**.** (1) For all real k_B and T, the Landauer calibration at k_B and T holds. (2) For any counter, there are two real-valued functions of the state that give 1 and 2 on every state with one unit, so the counter alone fixes no energy value.

Proof. (1) The scale says what one ledger unit is worth. At a ledger reading of one, multiplying by that scale returns the scale itself: $1 \cdot k_B T \ln 2 = k_B T \ln 2$. No condition on the signs of k_B or T is needed for that multiplication.

(2) Choose a scale of one and a scale of two. The same ledger reading of one is assigned energy one by the first choice and energy two by the second. The ledger has not changed; only the conversion I supplied has. Thus that conversion was not ■ determined by the counter.

The physics is the premise of Section 3, restated here once in the form the next theorems test. In words first: a *dissipation* hands each instruction a whole number of heat units, each unit worth kT ln 2. The floor premise says an instruction that takes at least one state where some yes-or-no property is false to a state where it’s true, never sends a true one back to false, and sends two different states to one, dissipates at least one unit. That’s a different statement from Landauer’s principle as Premise 3.5 has it. That one depends on the spread over states, $Q \geq k_B T \ln 2 \cdot \Delta H$, and gives no floor for a known state (there the distinction was absent from the chosen point distribution; that alone says nothing about a prior preparation bill). This one gives every such instruction a floor of one whole unit, whatever the spread. To get the second from the first you’d need more, at the least a spread with a full bit of uncertainty removed at every such step, and the known-state theorem and the narrow squeeze of Section 3 show that isn’t automatic. So I give it its own name, *structural dissipation pricing*. The calibration says no instruction dissipates more units than the counter charges it. One word about that word: “dissipation” here means heat handed to the surroundings, the quantity Section 3 read off Sagawa as owed even when no entropy is produced, the heat that pays for where the forgotten bit goes. It is not what Section 1 calls dissipating, which is entropy produced, and it has nothing to do with waste, or with doing the job badly. There are two premises here, nothing is proved about either, and the only question is whether they can hold together:

Premise 11.2 (Structural dissipation pricing)**.** A *dissipation* is a function δ from instructions to naturals, counted in units of $k_B T \ln 2$. The *floor*, a hypothesis separate from Premise 3.5 of Section 3: every instruction that collapses a yes-or-no property has $\delta(i) \geq 1$, where *i collapses P* when some state with P false is sent to a state with P true, every state with P true is sent to one with P true, and some two different states are sent to the same state. The *calibration*: $\delta(i) \leq \text{cost}(i)$ for every instruction. Neither is proved anywhere; the theorems below ask whether they can hold together.

The last clause of the floor is there because the first two don't make a step forget anything on a machine with infinitely many states. INC A sends a state with $A = 0$ to one with $A = 1$ and keeps every positive A positive, so it raises "$A \geq 1$" one way, and it is one-to-one. Any one-to-one map that misses some state does the same to the property "is in my image". Its input can be read back off its output, so it loses no entropy, and Landauer's premise as this book takes it then puts no heat floor under it at all. It pays in fresh states instead: INC moves every value of A up one, Hilbert's front desk, and only a counter with no top can keep doing it. A thermal implementation can dissipate under the model-specific conditions above; the premise only sets no floor under it. A floor read without the last clause would charge a whole unit. On a finite machine the clause asks nothing extra. A step that sends one false state into the true ones and keeps every true state true puts one more state into the true set than the set holds, so two of them land on the same state. And on any machine the property comes along for the ride: an instruction that sends two different states a and b to one collapses the property "is reached from a in one or more steps", since one of a and b lies outside it and lands in it. So the floor is a price on merging, one unit for every instruction that merges, read as heat.

11.2 The calorimeter protocol, as a specification

What follows is how one *would* measure this, and it's here because it makes the boundary exact. Take a two-state register with energies zero and a gap, a population p in the excited state, and the simplest possible rate equation for how p changes in one time step (a rate for climbing into the excited state, a rate for falling out of it, and the length of the step). For the canonical reset (population one half to zero, with the rates chosen so the equation closes), the heat delivered to the bath is exactly half the gap. Choose the gap to be twice kT ln 2 and that heat is exactly kT ln 2.

Read that twice, because the temptation is to read it as a derivation of Landauer's number and it's the opposite. The gap was *chosen* to make the heat come out as Landauer's number. The same populations and rates with half that gap give half the Landauer value, and there's a theorem exhibiting identical population dynamics

with different heat. The rate equation fixes the *shape*; the energy scale is a selected Hamiltonian (the physicist's name for the rule that says what energy each state has), and selecting it is not deriving it.

The canonical reset is a bookkeeping toy, and its rates show it. A register that trades heat with a bath at temperature T has rates in *detailed balance*: the rate up over the rate down is $e^{-\Delta/k_BT}$, which at the gap $2k_BT\ln 2$ is $1/4$. The canonical rates are 0 and 1, and the factor $e^{-\Delta/k_BT}$ is never 0, so no gap and no temperature put them in that ratio. With rates that are in it, a relaxation at a fixed gap moves toward the equilibrium population $1/(1+e^{\Delta/k_BT})$ and, with steps short enough not to overshoot, never goes below it. At the gap $2k_BT\ln 2$ that population is $1/5$. The toy has no work term either; nothing in it pays for the squeeze. Part (4) of the theorem below says what follows from that: at the gap $k_BT\ln 2$ the toy books less heat than Landauer's bound asks for the same squeeze from one half to zero, so the toy has no second law.

Definition 11.3 (The two-state protocol)**.** For a gap Δ, the register's mean energy at excited population p is $\Delta\cdot p$. One step of the rate equation is

$$p' = p + \delta t\,\big(k_{01}(1-p) - k_{10}\,p\big),$$

and the heat to the bath over a step from p to p' is defined as $\Delta p - \Delta p'$. The canonical reset takes $p=\frac{1}{2}$, $\delta t = 1$, $k_{01}=0$, $k_{10}=1$, ending at $p'=0$, and counts one logical event.

□ **Theorem 11.2** (The heat is fixed by the chosen gap)**.** (1) The canonical numbers satisfy the rate equation. (2) For every Δ, the canonical reset's heat is $\Delta/2$. (3) For all k_B, T, the gap $\Delta = 2k_BT\ln 2$ gives heat $k_BT\ln 2$. (4) If $k_B>0$ and $T>0$, the heat is below $k_BT\ln 2$ exactly when the gap is below $2k_BT\ln 2$; at the gap $k_BT\ln 2$ it is half of $k_BT\ln 2$. (5) There are two different gaps for which the same canonical populations satisfy the rate equation and the heats differ.

Proof. (1) Half the population starts excited. The upward rate is zero, so none enters the excited state; the downward rate is one for a step of length one, so the entire excited half leaves. Substitution gives $\frac{1}{2}+1\cdot(0-\frac{1}{2})=0$.

(2) The register began with mean energy $\Delta/2$ and ended with zero. The defined heat is their difference, $\Delta/2$.

(3) Choose $\Delta = 2k_BT\ln 2$. Half of that gap is $k_BT\ln 2$.

(4) Asking whether $\Delta/2 < k_BT\ln 2$ is the same as multiplying both sides by two and asking whether $\Delta < 2k_BT\ln 2$. At the particular gap $k_BT\ln 2$, part (2) gives half that value. With positive k_B, T it is strictly smaller because $\ln 2$ is positive.

(5) Take gaps one and two. The population equation in part (1) contains neither gap, so the same populations solve it in both cases. Their heats are one half and one respectively. The rate equation alone did not select between them. ■

A reset that pays its way is driven. Raise the gap a little, let the register settle with rates in detailed balance, raise it again, and so on, starting from a gap of zero, where the two states are alike and the population is one half. Raising the gap while the population holds is work done on the register; settling at the new gap is heat to the bath. That's the textbook way to wipe a bit slowly, and here it is with its numbers worked out.

Definition 11.4 (The driven reset)**.** Write kT for k_BT. At gap Δ the equilibrium excited population is $\pi(\Delta) = 1/(1+e^{\Delta/kT})$ and the register's free energy is $F(\Delta) = -kT\ln(1+e^{-\Delta/kT})$. Rates satisfy *detailed balance* at gap Δ when $k_{01} = k_{10}\,e^{-\Delta/kT}$. A *driven reset* along a schedule of gaps $g_0, g_1, \ldots, g_n$ takes, at step m, the gap from g_m to g_{m+1} with the population held at $\pi(g_m)$, which is work $\pi(g_m)(g_{m+1}-g_m)$, and then lets the register settle to $\pi(g_{m+1})$, which is heat $g_{m+1}\big(\pi(g_m)-\pi(g_{m+1})\big)$ to the bath. The reset's work and heat are the sums over its steps. The uniform schedule from 0 to D in N steps has $g_m = Dm/N$.

Theorem 11.3 (A driven reset pays $kT\ln 2$ in work)**.** (1) With rates in detailed balance at gap Δ, one step of the rate equation takes p to $\pi(\Delta) + \big(1-\delta t\,(k_{01}+k_{10})\big)\big(p-\pi(\Delta)\big)$; when $\delta t\,(k_{01}+k_{10}) = 1$ this discrete update lands on $\pi(\Delta)$ from any p. (2) For any schedule, work minus heat is $g_n\pi(g_n) - g_0\pi(g_0)$, the change in the register's mean energy. (3) If $kT > 0$ and the schedule never falls, the work is at least $F(g_n) - F(g_0)$; if also every step raises the gap by at most h, the work is at most $F(g_n) - F(g_0) + h\big(\pi(g_0) - \pi(g_n)\big)$. (4) If $kT > 0$, $D \geq 0$ and $N \geq 1$, the uniform reset from 0 to D in N steps does work between $kT\ln 2 - kTe^{-D/kT}$ and $kT\ln 2 + D/(2N)$, and leaves an excited population of at most $e^{-D/kT}$. □

Proof. (1) Detailed balance gives $k_{01} = (k_{01}+k_{10})\pi(\Delta)$. In the rate equation, the upward contribution minus the downward one is $k_{01} - (k_{01}+k_{10})p$. Substitute the balance equation and this becomes $-(k_{01}+k_{10})(p-\pi(\Delta))$. The update therefore subtracts the fraction $\delta t(k_{01}+k_{10})$ of the distance from equilibrium. What remains is exactly

$$p' = \pi(\Delta) + \big(1-\delta t(k_{01}+k_{10})\big)\big(p-\pi(\Delta)\big).$$

When that fraction is one, none of the distance remains: $p' = \pi(\Delta)$.

(2) Follow one raise from gap a to gap b. Its work is $\pi(a)(b-a)$ and its heat is $b(\pi(a)-\pi(b))$. Subtract the heat from the work. The two occurrences of $b\pi(a)$ cancel, leaving $b\pi(b)-a\pi(a)$, the final mean energy minus the initial mean energy. At the next raise, that final energy becomes the next initial energy. Adding over the schedule cancels every intermediate energy and leaves only $g_n\pi(g_n)-g_0\pi(g_0)$.

(3) Differentiating the displayed free energy gives slope $F'(\Delta)=\pi(\Delta)$. With $kT>0$, this slope decreases as the gap rises. Over $a\le b$, the increase of F therefore lies between the widths times the two endpoint slopes:

$$\pi(b)(b-a)\le F(b)-F(a)\le \pi(a)(b-a).$$

The work uses the right-hand quantity, the slope before the raise. It is at least the free-energy increase. Its excess is at most $(b-a)(\pi(a)-\pi(b))$. If the raise is at most h, that excess is at most $h(\pi(a)-\pi(b))$.

Add those bounds over the schedule. Intermediate free energies cancel, and intermediate populations cancel in the excess bound. I obtain work between $F(g_n)-F(g_0)$ and that amount plus $h(\pi(g_0)-\pi(g_n))$.

(4) For the uniform schedule, $h=D/N$, $\pi(0)=1/2$, and

$$F(D)-F(0)=kT\ln 2-kT\ln(1+e^{-D/kT}).$$

For $x\ge 0$, $0\le \ln(1+x)\le x$: it starts at zero and its slope $1/(1+x)$ is between zero and one. The free-energy difference is thus at least $kT\ln 2-kTe^{-D/kT}$ and at most $kT\ln 2$. The excess from (3) is at most $(D/N)(1/2)$, giving the work bounds. Finally the denominator of $\pi(D)=1/(1+e^{D/kT})$ is at least $e^{D/kT}$, so the remaining excited share is at most $e^{-D/kT}$. ■

Try one finite step in units where $kT=1$: start with gap zero and half the registers excited, then raise the gap to $\ln 2$. The work is $\frac{1}{2}\ln 2$. After settling, the excited share is $1/3$, so the bath receives $(\frac{1}{2}-\frac{1}{3})\ln 2=\frac{1}{6}\ln 2$ and the registers retain $\frac{1}{3}\ln 2$ more energy. Work equals those two amounts added together. A third are still excited: one such step has not wiped the bit.

The word “step” matters here. Part (1) is a discrete update of the population. Its multiplier can be zero. Continuous relaxation at fixed positive rates instead multiplies the distance from equilibrium by $e^{-(k_{01}+k_{10})t}$, which is positive at every finite time. Exact settling in the schedule is an idealisation; the algebra does not prove that a continuous bath completes it in one finite waiting period.

Here nobody picked the gap to make the number come out. Take D large and N much larger than D: the work closes in on $kT\ln 2$ from both sides, and the population left in the excited state goes to zero. Perfect erasure is the limit, not the result at any finite gap. To reuse the original symmetric memory, the gap must also return to zero while transitions are suppressed. Lowering it at the final fixed

population recovers $D\pi(D)$ of work and leaves that population unchanged. This term tends to zero in the same limit. The reset of the population, restoration of the trap and time needed for settling are separate parts of the physical protocol. Detailed balance supplies the thermal premise; the displayed sums supply the work and heat accounting for the idealised schedule.

What ties it to a machine is that the protocol counts one logical event, by definition, and one is what the latch's raising move costs, the least the toll lets it cost. The tie is two numbers that happen to match, the protocol's one by definition and the machine's one by the toll, and nothing more. No theorem maps the protocol's two register states onto any state of the machine. And "heat to the bath" here means the register's mean energy before minus after, which quietly assumes the bath gets exactly what the register loses; nothing here establishes that. So a device test would have to measure the gap, the populations, the rates, and the bath heat independently; separately establish that the device's rates are in detailed balance (that the up and down rates stand in the ratio the temperature dictates), which the driven reset assumes and the canonical one breaks; and separately establish that the device's physical reset *is* the machine's certify event. The driven reset is an executable experimental specification. The canonical reset is a bookkeeping toy. Neither is a measured device, and neither is an unconditional conversion of units of cost to joules, and anyone who tells you otherwise has skipped the conditions.

11.3 What this section is

This section holds a calibration that's a unit conversion; a pair of premises, consistent on the steps the toll is about and inconsistent on the whole instruction set; a history that makes any step injective by paying with a fresh state at every step; and a measurement protocol that's a specification with a selected scale. The physics in this book is in Part I: Landauer's principle as a named premise with its critics named (Section 3), and conditional finite-time dissipation bounds, cited, and proved for one model (Section 1). Here there's accounting, and I've tried to call every line by its right name. That's the end of the example machine. What's left, after one suspicion I can't prove, is how you'd break any of this, and where it stops.

CHAPTER 12

From “The claims and their conditions”

12.1 The diagonal and the worked models

The diagonal argument supplies the contradiction inside a substrate that can represent the candidate’s flip and has the stated recursion property. Its fixed point makes the proposed decision disagree with itself. An outside calculation of the natural-number example does not supply an inside program for that substrate. Section 14 and the appendices state the input conventions separately for the natural-number family and L.

The protocol examples in Section 16 isolate particular mechanisms: finality, a charge on commitment, an attestation window, an inclusion proof and a bounds check. Their statements follow the operations in those fragments. The pointer thesis then asks why certification is the event to choose. It proposes a connection to records copied and relied on by independent parties. The counting and window arguments apply to any record meeting their premises; the thesis supplies the proposed reason for this choice.

Those connections are what the book has built: a count, a schedule, an observation and a record that stays.

CHAPTER 13

Methodology note

Everything this book claims as its own stands or falls on the arguments written in it; what it borrows is cited to the people who proved it, and the physics enters as a named premise. If a proof on the page is wrong, the result is wrong, and nothing else I could tell you would rescue it.

Language models were my hands, as I said in Section 23: they did symbol-pushing, drafted code and helped work through derivations. I have to answer for what stayed. Start with the definitions and follow how each step leads to the conclusion. If I counted a finite construction, its cases belong on the page. If I claimed something for every run, the argument has to explain why the next step preserves it. A successful example cannot carry that claim, and a name attached to a proof cannot fill a missing step.

The same demand applies to the picture around the mathematics. A definition has to say what I meant, and a model of a device has to fit that device. Those questions belong beside the premises and the argument, where you can examine the fit.

CHAPTER 14

Rows of the claim ledger for the passages above

The book's claim ledger listed these claims, each with the kind of claim it is and where it lived.

Claim	Kind	Home
A decision tree has at most two to its depth leaves	definition; proved	Section 10, Definition 4.1, Lemma 4.1
Paid narrowing, counted	proved with the total of the charges, which equals the counter's rise by conservation; both are proved (premises: a covering tree, paid steps)	Section 10, Theorem 4.2
Structural entitlement with the narrowing made by the run, over any Thiele machine whose test steps are named: every drop made by a test the run passed, the exact product count, index bits at most k times the cost	proved (premises: test steps charged at least one, a prior containing the start, a 2^{-k} share kept by each test)	Section 10, Theorem 4.3
The latch on a Turing machine, a register machine, L	definition; proved (and the relation is an equivalence)	Section 12, Definition 5.2, Theorem 5.1

Claim	Kind	Home
Halving pricing bounds weighted conditional entropy; an infinite landing pile admits no finite halving price	proved; average loss still depends on the distribution	Section 13, Theorems 6.1 and 6.2
The nat family: each candidate decider misses program two in the instance built from it	proved	Section 14, Theorem 7.1
Recursion, Rice and halting for L	definition; proved	Section 14, Definition 7.1, Theorem 7.2
Platform and quote; inclusion check; bounds fragment	definition	Section 16, Definition 16.4, Definition 16.5, Definition 9.1
The proof-carrying checker is the verification condition	proved	Section 16, Theorem 9.1
Ecosystem, proliferation, pointer; copied and relied on, independent carriers; observation game	definition	Section 17, Definition 10.1, Definition 10.2, Definition 10.3
Independent copies on every state, with two observers or more: the event is equivalent at any two states	proved	Section 17, Theorem 10.1
In the specified replicated-header toy, gas used and finality are copied, and the chosen action maps rely only on finality	proved for those maps	Section 17, Theorem 10.2
In the toy, “certified” is the unique pointer	proved	Section 17, Theorem 10.3
The survey; the swap shows the verdict follows the observer maps	proved	Section 17, Theorem 10.4

Claim	Kind	Home
One blind observer stops proliferation	proved	Section 17, Theorem 10.5
Agreement doesn't buy permanence; durable views that report the event do	proved	Section 17, Theorem 10.6
The calibration is a unit conversion	definition; proved	Section 18, Definition 11.1, Theorem 11.1
Structural dissipation pricing (a whole-unit floor on collapsing instructions, separate from Landauer's principle); dissipation at most cost	assumed (premise)	Section 18, Premise 11.2
The heat is fixed by the chosen gap	definition; proved	Section 18, Definition 11.3, Theorem 11.2
A driven reset with rates in detailed balance, its gap raised from 0 to D in N equal steps, pays between $kT\ln 2 - kTe^{-D/kT}$ and $kT\ln 2 + D/(2N)$ in work	definition; proved	Section 18, Definition 11.4, Theorem 11.3
Structural dissipation pricing	every collapsing instruction dissipates at least one unit, and dissipation is at most cost	Section 18, Premise 11.2
Covering tree, paid steps	a decision tree whose leaves cover the narrowing, and at least as many steps charged at least one as the tree is deep	entitlement, Theorems 4.2 and 10.1
Share kept by a test	each passed test keeps at least a 2^{-k} share of the candidates, for a k fixed before the run; every run meets it for k its own biggest test	entitlement made by the run, Theorem 4.3

Claim	Kind	Home
The canonical reset's rates aren't in detailed balance at any gap or temperature; with rates that are, a fixed-gap relaxation moves toward $1/(1+e^{\Delta/k_BT})$, which is $1/5$ at the gap $2k_BT\ln 2$	proved	Section 18

CHAPTER 15

From the plain-text statement

These passages were taken out of `THIELE_MACHINE.txt` and are kept here as they stood.

```
BUILD THE BOX, THEN FORGET WHAT THE TASK DOES NOT ASK FOR

The smallest machine above an ordinary computing box has a lamp beside it
and a tally under it. Computing buttons follow the old rule, cost nothing,
and leave the lamp and tally alone. A mark leaves the computing picture
alone, adds one to the tally, and turns the lamp on if its named event holds.
Once on, it stays on. Begin with the lamp down and the tally at zero.

Run only computing buttons and the old machine runs exactly as before.
Forget the lamp and tally afterward and you recover its computation.
The larger description contains it. An event that holds somewhere gives
a visible difference between before and after marking even when the
computing picture stays the same. An ordinary computer can store the
picture, lamp and tally when their rules are effectively given.

Now let two builders implement those instructions differently. One keeps
every button press on a tape. The other uses a few extra counters. Ask
each for the computing picture, lamp and exact tally. Require computing
to follow the base rule, marks to latch exactly the named event without
changing the picture, and each move to add its zero-or-one charge to the
tally. The reading must obey the toll and permanence.

Read those three things from either implementation. After a computing
move, the picture follows the old rule; the free move cannot change the
permanent lamp; adding zero leaves the tally alone. After a mark, the
picture stays put, the lamp latches the event and the tally rises by one.
Those are precisely the simple box's rules. Repeat this for every button
in a list and the readings agree for the whole run. Any list reaching a
state of the simple box reaches an implementation state with those same
readings. Starting together gives all the same reachable triples.

That makes three stages: write a button list, run it on the builder's
```

box, then read the picture, lamp and tally. The last step forgets whatever extra history the builder kept. There is only one way to do it while preserving those three readings: report their actual values.

Try running that last step backward. Turn the simple box halfway around, then turn it again. The picture is back, and the lamp and tally never changed. Give the other box a separate counter that counts free turns. Its counter is now two larger. Which counter value should the simple box's one state turn into? Preserving the turns would require both. There is no such map. Forgetting the extra counter works; recovering it from the three readings does not.

To see what has to be kept, give two states the same future button list. If a required answer differs, those states cannot share one full state in a faithful implementation. That one state would have to give two answers to the same continuation. Include the empty list: reading now counts as a test. Group states only when every such test agrees. Once grouped, pressing a common button keeps them grouped, because that button can be put in front of every remaining test. Every grouping that preserves answers and steps must respect these tests. This is the coarsest one.

For the picture, lamp and tally, equal current readings already give identical future readings: their next values follow exactly the rules above. Different triples are distinguished by reading them now. Remove states that differ only in history these questions can never reveal, and every reachable triple has exactly one remaining state. Any two implementations with no such redundancy differ only in the names of those states. A button list reaching a state fixes its partner in the other implementation, so even that renaming is forced.

The questions do work here. Ask for the extra turn count and it has to survive too. Stop asking for the tally and repeated failed marks can become indistinguishable. If no mark event ever holds, a clean run never reaches a lamp-up state. The minimum concerns what can be reached and what the chosen questions can distinguish. It does not select the event. I develop this familiar idea from minimal automata for the actual lift, with the observations and accounting kept explicit.

Put a hidden switch behind a dark lamp. Pressing the button copies the switch to the lamp. Both boxes can look dark now and give different next answers. Keep only the lamp and the next step cannot be predicted from that picture. Keep the switch too and it can. The exact test is proved: equal pictures must g pictures under every permitted use of the same button, with explicit representatives and successor choices for the converse construction.

The arrows do not always choose which question matters. Nor does a finite watch prove that a lamp stays lit forever: a reset can sit just past the last tick anyone watched. These boundaries have checked counterexamples. Showing that real equipment follows the chosen model needs evidence about that equipment as well as a proof about the model.

The first two counts concern tasks with their own stated operations and recovery requirements. The third concerns a publication-file extension of the counter example. They count fibres of different projections.

First ask which checks passed on the current page. At working value b, let m(b) named predicates be true. Allow each successful check to be requested independently. Require an exact answer for every name since the last WRITE. One check gives two choices: run it or leave it alone. A second independent check doubles the choices, and each further check doubles them again. With m(b) available checks there are 2^m(b) possible selections. Realize any subset by writing b and running exactly those checks. Two different subsets cannot end at the same state because one named query would need opposite answers. The fibre above b therefore needs at least 2^m(b) states.

That bound is attainable. Use states (b,A), where A is a subset of the true check names at b. WRITE replaces b and clears A. A passing CHECK inserts its name. A failed CHECK and IDLE preserve the state. The empty start is correct; every command preserves the intended history reading. These are all the command cases, so the claim holds for every finite program. Every legal state is reachable by the realizing word already given. For finite B, the exact total is the sum of 2^m(b) over its working values.

Next keep the answers across wiped editions. On each edition, publish or leave it alone. Th two histories, the second four, the third eight. If n independent publication choices must remain readable when the workspace returns to one picture b, there are 2^n distinguishable histories above that picture. For one machine that must do this for every n, no finite fibre suffices: given any proposed size K, choose n with 2^n>K. That is an infinite reachable fibre forced by unlimited, recoverable edition history.

CHAPTER 16

From the README

These paragraphs were taken out of the repository README and are kept here as they stood.

```
The smallest machine standing above an ordinary computing box has a lamp beside it and a tall

Another builder can keep an extra tape or counter. Require the same base steps, exact event l

The construction goes from **button lists, to implementation states, to picture, lamp and tal

Which differences must survive? Give two states the same future button list. If any required

Ask for the extra turn count and more has to survive. Stop asking for the tally and less may

Put a hidden switch behind a dark lamp. The next button press copies the switch to the lamp.
```

CHAPTER 17

From the plain-text statement and the README, second pass

These passages were taken out of `THIELE_MACHINE.txt` and the README when the two were rebuilt as the argument, and are kept here as they stood.

```
WHAT A FREE MOVE CAN CHANGE

Slide a tile to the right without paying. Ask whether it is in the place
it just reached. The answer was no and is now yes. If the toll charged
every question, that slide could not be free. Do this with any changed
full state: ask whether it is the destination. Where states can be tested
for equality, charging every Boolean question would freeze every free
move. The definition has to name the reading its toll belongs to.

Now keep that reading permanent. A free move cannot raise it, because a
raise costs at least one. It cannot lower it, because a yes stays yes.
It must leave it alone. That explains the reading part of the horizontal
rule. The window adds another requirement: the next picture comes from
the present picture and the button pressed. Permanence by itself does
not choose a picture or make a paid move leave that picture alone.

There are two ways to state the same reading requirement for a sliced
machine. Leave it alone on horizontal moves and never lower it on vertical
moves, and it is permanent through all moves. Start instead with permanence,
the toll and free horizontal moves, and the two sentences above force
horizontal preservation. Neither route has to assume its own conclusion.


PUT THE PRICE ON THE BUTTON

Put price cards zero, one and two beside a button. If pressing it can raise
the lamp from some state, zero fails there. One works. Two works too, but
one is the least. If the button never raises the lamp from any state,
zero works. Give each button its least card and you have the least valid
schedule, one instruction at a time. Any other least schedule must have
```

the same cards: neither can be smaller than the other.

For a finite machine, inspect a list covering all its states to decide which card each button needs. On an infinite machine that search need not finish. For the lift there is a direct witness: if each mark's event holds somewhere, put the lamp down over that base state and press the mark. It raises the lamp, so one is necessary. Computing leaves the lamp alone, so zero suffices. A mark whose event never holds needs no positive toll; charging one for every use then comes from the chosen accounting rule.

The price stays on the instruction. It does not disappear when you press the same button with the lamp already on. Every run from no to yes contains a crossing and pays at least one; conversely a single crossing is already a run. The step toll and the floor from every start give the same minimum. That is a minimum in the definition's units. Measuring a device's heat requires a physical account of the device.

The allowed moves and the starting states matter. Correctly translating a program proves a simulation. To preserve a guarantee through every allowed host program, the proof must cover every host move from the stated starts. Paying to raise a record does not, by itself, establish what that record means.

This does not require the full states to form a literal product. It only requires the two readings and the displayed equations. If someone reads the hidden record and chooses a different instruction because of it, the two executions no longer receive the same horizontal word.

For the entry arrow u -> f(u) itself to share its destination with an arrow from H, the exact condition is f(u) in f(H). Surjectivity of f on H is sufficient, as is a single holding state. Neither is necessary for a particular entry to collide.

These are full configurations. A fixed array of bits can have many different configurations without acquiring a new memory cell on every tick.

A different accounting interface allows a source-dependent charge a(s), or c(s,i) when instructions are supplied. This extends the instruction-price definition above; it does not change the meaning of c(i) there. Its toll must be stated on each actual crossing. On no -> yes -> yes, charge one at no and zero at yes. A ledger with values zero and one obeys both updates. If instead every ti positive charge, a fixed full state could not hold its own ledger: it would require the same number to equal itself plus a positive amount.

```
The mathematics specifies the operations that must be preserved. A software
implementation must implement those operations through every allowed path.
Keeping a copy of their names or values alone does not establish that.

These bounds follow the information available and the question being asked.
An ordinary program given the live question can accumulate its answer too.
The earned machine adds the requirement that its permitted operations keep
the accepting record faithful to what actually passed.


Every machine with a run that ends on a state other than its start, and states you can test

## Which differences have to stay?

Slide a tile to the right for free. Ask whether it is in the place it just reached. The ans

Keep that reading permanent. A free move cannot raise it, because a raise pays. It cannot l

Put price cards zero, one and two beside a button. If it can raise the reading from some st

Now put a price rule beside the arrows. The abstract model's toll says that a move raising

Storing a representation is part of implementing this machine. Preserving those rules throu

The machine can be implemented with ordinary computation. What matters is which information
```

CHAPTER 18

Further reading

The works below were read for the book and are not named in its finished text or in this volume. They are kept here with their annotations, so that nothing read for the book is lost.

18.1 Structure, information and knowledge

Hans Albert. Wikipedia. Article. German philosopher (1921–2023), with Popper a leading critical rationalist; Traktat über kritische Vernunft (1968).

A. J. Ayer (1956). The Problem of Knowledge. Macmillan, London, p. 34. The other definition Gettier targets.

Max Black. Wikipedia. Article. The Russian-born British-American philosopher, at Cornell from 1946, and his 1952 paper on two spheres.

Roberto Bonola (1912). Non-Euclidean Geometry: A Critical and Historical Study of Its Development. Open Court; Dover reprint, 1955. The attempts on the parallel postulate from Proclus through the Islamic geometers and Saccheri, and Farkas Bolyai's letter to his son.

G. K. Chesterton (1929). The Thing. The fence passage is quoted in the American Chesterton Society's Lecture 57: The Thing.

Gösta Grahne (1998). Updates and counterfactuals. Journal of Logic and Computation 8(1), pp. 87–117. DOI: 10.1093/logcom/8.1.87. Revision of a static world versus update of a changing one.

Jonathan Jenkins Ichikawa and Matthias Steup (2001, substantive revision 2026). The Analysis of Knowledge. Stanford Encyclopedia of Philosophy. Entry. Dharmottara's mirage and Peter of Mantua's running Plato.

Rosalie Iemhoff (2008, substantive revision 2019). Intuitionism in the Philosophy of Mathematics. Stanford Encyclopedia of Philosophy. Entry. Kreisel's creating-subject axioms CS1–CS3 (1967), CS2 being that the creating subject never forgets what it has proven, and Troelstra's strengthening (1969).

Petrus Mantuanus (d. 1400). Logica. Medieval manuscripts in Oxford libraries, Bodleian Library. Catalogue entry. Peter of Mantua as a logician of the late fourteenth century.

Per Martin-Löf (1996). On the Meanings of the Logical Constants and the Justifications of the Logical Laws. Nordic Journal of Philosophical Logic 1(1), pp. 11–60. Lectures given in Siena in 1983. The passage on proof as an act is on p. 28. Lecture text.

Meta Platforms (2021). Introducing Meta: A Social Technology Company. Company announcement, 28 October 2021. Announcement.

Benjamin Morison (2014, revised 2026). Sextus Empiricus. Stanford Encyclopedia of Philosophy. Entry. The source for his dates, his medicine and his skepticism.

J. J. O'Connor and E. F. Robertson (n.d.). Theaetetus of Athens. MacTutor History of Mathematics. Entry.

J. J. O'Connor and E. F. Robertson (n.d.). Paul Richard Halmos. MacTutor History of Mathematics. Biography.

Benoît Rittaud and Albrecht Heeffer (2014). The Pigeonhole Principle, Two Centuries before Dirichlet. The Mathematical Intelligencer 36(2), pp. 27–29. Jean Leurechon's use of the principle in 1622.

Göran Sundholm (1998). Proofs as Acts and Proofs as Objects: Some questions for Dag Prawitz. Theoria 64(2–3), pp. 187–216. DOI: 10.1111/j.1755-2567.1998.tb00187.x.

Göran Sundholm (2014). Constructive Recursive Functions, Church's Thesis, and Brouwer's Theory of the Creating Subject: Afterthoughts on a Parisian Joint Session. In Constructivity and Computability in Historical and Philosophical Perspective, Logic, Epistemology, and the Unity of Science, Springer, pp. 1–35. DOI: 10.1007/978-94-017-9217-2_1.

Timothy Yenter and Ezio Vailati (2003, revised 2024). Samuel Clarke. Stanford Encyclopedia of Philosophy. Entry. Clarke as the British philosopher and theologian who defended Newton's natural philosophy in his letters with Leibniz.

18.2 Cost, records and thermodynamics

János Aczél, Bruno Forte and C. T. Ng (1974). Why the Shannon and Hartley entropies are "natural." Advances in Applied Probability 6(1), pp. 131–146. DOI: 10.2307/1426210. Ng's 1971 Waterloo doctorate under Aczél: Mathematics Genealogy Project.

Roberto M. Amadio, Nicolas Ayache, François Bobot, Jaap P. Boender, Brian Campbell, Ilias Garnier, Antoine Madet, James McKinna, Dominic P. Mulligan, Mauro Piccolo, Randy Pollack, Yann Régis-Gianas, Claudio Sacerdoti Coen, Ian Stark and Paolo Tranquilli (2014). Certified Complexity (CerCo). Foundational and Practical Aspects of Resource Analysis, FOPARA 2013, LNCS 8552, pp. 1–18. DOI: 10.1007/978-3-319-12466-7_1.

Walter J. Boyne. Wikipedia entry: United States Air Force officer, aviation historian, director of the National Air and Space Museum (acting from 1982, director 1983–1986). Entry.

Léon Brillouin (1951). Maxwell's Demon Cannot Operate: Information and Entropy. I. Journal of Applied Physics 22(3), pp. 334–337. DOI: 10.1063/1.1699951.

Arthur Charguéraud and François Pottier (2019). Verifying the Correctness and Amortized Complexity of a Union-Find Implementation in Separation Logic with Time Credits. Journal of Automated Reasoning 62, pp. 331–365; first published online in 2017. DOI: 10.1007/s10817-017-9431-7.

Gavin E. Crooks (1999). Entropy production fluctuation theorem and the nonequilibrium work relation for free energy differences. Physical Review E 60(3), pp. 2721–2726. DOI: 10.1103/PhysRevE.60.2721. A path's chance against its reverse's is e to the dissipated work.

Joe DeBlasio (2025). Discussion: removal of one-6962-log requirement. Chromium ct-policy mailing list, 17 November 2025. Post.

Jérémie Decouchant, David Kozhaya, Vincent Rahli and Jiangshan Yu (2022). Damysus: streamlined BFT consensus leveraging trusted components. Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys '22), pp. 1–16. DOI: 10.1145/3492321.3519568.

Encyclopaedia Britannica. Donation of Constantine. Web page. Composed in the eighth century; the consensus view puts it in the 750s or 760s.

Encyclopaedia Britannica. Leo Szilard. Web page. Born in Budapest in 1898; Berlin doctorate 1922 and the Institute of Theoretical Physics there.

Ethereum Foundation (n.d.). The Merge. ethereum.org roadmap page, on the transition of 15 September 2022. Page.

Ethereum Foundation (n.d.). Timeline of all Ethereum forks (Tangerine Whistle, EIP-150, and the Berlin upgrade, block 12,244,000). ethereum.org. Page.

Ethereum Foundation. Devcon editions (Devcon 0, Berlin 2014; Devcon 1, London 2015; Devcon 2, Shanghai 2016). devcon.org. Page.

Ethereum Foundation. Ethereum execution specifications, Prague fork, src/ethereum/forks/prague/vm/instructions/storage.py and vm/gas.py, at commit ecb68f21a7d28db101c769878184f6facb319da1. Repository.

Federal Aviation Administration (2008). Revisions to Cockpit Voice Recorder and Digital Flight Data Recorder Regulations. Final rule, Federal Register 73(46), 7 March 2008, pp. 12542–12573. The two-hour cockpit voice recorder.

Federal Aviation Administration (2026). 25-Hour Cockpit Voice Recorder (CVR) Requirement, New Aircraft Production. Final rule, Federal Register 91(21), 2 February 2026, pp. 4447–4462. Federal Register text. The thirty-minute recorders of 1966, the two-hour rule, and the 25-hour standard of ICAO (2016) and EASA (from 2021).

Richard FitzNigel (c. 1180). Dialogus de Scaccario (The Dialogue of the Exchequer). Text and translation ed. Charles Johnson, Nelson's Medieval Classics, 1950. The widths of the cuts on Exchequer tallies.

Fox-IT (2012). Black Tulip: Report of the investigation into the DigiNotar Certificate Authority breach. Version 1.0. Report.

Tom Furtwangler (2017). The vaccine vial monitor: the world's smartest sticker. PATH, 6 November 2017. Article. The stickers first sold in 1996, for oral polio vaccine.

General Conference on Weights and Measures (2018). Resolution 1 of the 26th meeting: On the revision of the International System of Units (SI), effective 20 May 2019.

C. H. Jenkinson (1910). Early wooden tallies relating to Surrey. Surrey Archaeological Collections 23, pp. 203–207. PDF. Hazel; the payer kept the stock, the Exchequer the foil; abolished by statute in 1826; the burning that started the fire.

Joint Committee for Guides in Metrology (2012). International vocabulary of metrology: basic and general concepts and associated terms (VIM), 3rd edition, JCGM 200:2012, entry 2.41, "metrological traceability." The definition is on p. 29; "does not ensure . . . that there is an absence of mistakes" is Note 5 to 2.41, p. 30. BIPM text.

Mikhail Kuter, Marina Gurskaya and Albert Gurskii (2025). Genoa 1340, Double Entry, and 'Gold Coasting'. De Computis, Revista Española de Historia de la Contabilidad 22(1), pp. 1–33. DOI: 10.26784/issn.1886-1881.22.1.11927.

Klaus-Jörn Lange, Pierre McKenzie and Alain Tapp (2000). Reversible Space Equals Deterministic Space. Journal of Computer and System Sciences 60(2), pp. 354–367. DOI: 10.1006/jcss.1999.1672.

Ben Laurie, Adam Langley and Emilia Kasper (2013). Certificate Transparency. RFC 6962. RFC 6962; DOI: 10.17487/RFC6962.

Ben Laurie, Eran Messeri and Rob Stradling (2021). Certificate Transparency Version 2.0. RFC 9162. RFC 9162; DOI: 10.17487/RFC9162.

A. Liapounoff (1907). Problème général de la stabilité du mouvement, traduit du russe par Édouard Davaux. Annales de la Faculté des Sciences de Toulouse, 2e série, 9, pp. 203–474. Numdam. The French translation the 1992 English one was made from.

Sinisa Matetic, Mansoor Ahmed, Kari Kostiainen, Aritra Dhar, David Sommer, Arthur Gervais, Ari Juels and Srdjan Capkun (2017). ROTE: rollback protection for trusted execution. Proceedings of the 26th USENIX Security Symposium, pp. 1289–1306. USENIX page.

Clyde Lee Miller (2009; revised 2021). Cusanus, Nicolaus [Nicolas of Cusa]. Stanford Encyclopedia of Philosophy. SEP entry.

Yûto Murashita, Ken Funo and Masahito Ueda (2014). Nonequilibrium equalities in absolutely irreversible processes. Physical Review E 90, 042110. DOI: 10.1103/PhysRevE.90.042110. The bound on average entropy production by $-\ln(1-\lambda)$, with free expansion as the first example.

Satoshi Nakamoto (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. Paper.

Devon O'Brien (2018). Certificate Transparency Enforcement in Google Chrome. Chromium ct-policy mailing list, 6 February 2018. Announcement.

Garson O'Toole (2016). There Ain't No Such Thing as a Free Lunch, TANSTAAFL. Quote Investigator, 27 August 2016. Article.

Pavel Pakshin, Elena Panteley and Natalia Pakshina (2017). The Legacy of A. M. Lyapunov to Systems and Control Theory: 110 years of the Toulouse edition. IFAC World Congress 2017, open-track proposal. Davaux's translation, reviewed and corrected by Lyapunov.

Lídia del Rio, Johan Åberg, Renato Renner, Oscar Dahlsten and Vlatko Vedral (2011). The thermodynamic meaning of negative entropy. Nature 474, pp. 61–63. DOI: 10.1038/nature10123; arXiv: 1009.1630.

Jürgen Schnakenberg (1976). Network theory of microscopic and macroscopic behavior of master equation systems. Reviews of Modern Physics 48, pp. 571–585. DOI: 10.1103/RevModPhys.48.571. The entropy a Markov chain hands its surroundings, as the net rate across each pair times the log of the forward-to-backward rate ratio, which at a steady state is the whole entropy production; the biased walk is its smallest case.

Securities and Exchange Commission (2022). Electronic Recordkeeping Requirements for Broker-Dealers, Security-Based Swap Dealers, and Major Security-Based Swap Participants. Release No. 34-96034, 12 October 2022. Final rule. Adds an audit-trail alternative to the write-once requirement of Rule 17a-4(f) in place since 1997.

Caroline Shenton (2012). The Day Parliament Burned Down. Oxford University Press. The tallies burned in the furnaces under the House of Lords, 16 October 1834.

Michael Stock, Pauline Barat, Richard S. Davis, Alain Picard and Martin J. T. Milton (2015). Calibration campaign against the international prototype of the kilogram in anticipation of the redefinition of the kilogram part I: comparison of the international prototype with its official copies. Metrologia 52(2), pp. 310–316. DOI: 10.1088/0026-1394/52/2/310.

Wei Tang (2019). EIP-2200: Structured Definitions for Net Gas Metering. Ethereum Improvement Proposals. eips.ethereum.org/EIPS/eip-2200.

This Day in Aviation (n.d.). 30 October 1935: Boeing Model 299 crash at Wright Field. Page.

Kent Weare (2016). Ethereum Security Alert Issued, Ethereum Foundation Responds with “From Shanghai, With Love”. InfoQ, 19 September 2016. Article.

World Health Organization (2011). PQS performance specification: Vaccine Vial Monitor. WHO/PQS/E06/IN05.2.

18.3 Computation, self-reference and checking

Keith Adams and Ole Agesen (2006). A Comparison of Software and Hardware Techniques for x86 Virtualization. Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XII), pp. 2–13. DOI: 10.1145/1168857.1168860. Binary translation of instructions such as POPF, which on x86 runs in user mode without trapping. Intel’s VT-x and AMD-V in 2005 and 2006: Wikipedia, x86 virtualization.

American Standards Association (1963). American Standard Code for Information Interchange, ASA X3.4-1963.

Henk Barendregt. Wikipedia. Article. A Dutch logician known for his work in lambda calculus and type theory.

Gerd Behrmann, Ansgar Fehnker, Thomas Hune, Kim Larsen, Paul Pettersson, Judi Romijn and Frits Vaandrager (2001). Minimum-cost reachability for priced timed

automata. Hybrid Systems: Computation and Control (HSCC 2001), Lecture Notes in Computer Science 2034, pp. 147–161.

N. G. de Bruijn (1972). Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church–Rosser theorem. Indagationes Mathematicae (Proceedings) 75(5), pp. 381–392. DOI: 10.1016/1385-7258(72)90034-0. Variables written as numbers that count the binders back, now called de Bruijn indices.

Stanley Burris and Marcel Jackson (2010, substantive revision 2026). George Boole. Stanford Encyclopedia of Philosophy. Entry.

Stanley Burris and Javier Legris (2009, substantive revision 2021). The Algebra of Logic Tradition. Stanford Encyclopedia of Philosophy. Entry.

G. K. Chesterton. Wikipedia. Article. An English writer, referred to as the “prince of paradox”.

Donald W. Davies (2004). Corrections to Turing’s Universal Computing Machine. In B. Jack Copeland (ed.), The Essential Turing, pp. 103–124. Oxford University Press. Davies at the National Physical Laboratory from 1947 and the word *packet*: Wikipedia, Donald Davies.

Charles Dickens (1843). A Christmas Carol. Chapman and Hall, London. Stave Three, the game of Yes and No.

William Henry Eccles and Frank Wilfred Jordan (1920). Improvements in ionic relays. British patent GB148582A, filed 21 June 1918, published 5 August 1920. Patent text. The 1918 date in the text is the filing date. The circuit as the flip-flop, the basis for electronic memory: History of Information, entry; and Wikipedia, Flip-flop (electronics), on the Eccles–Jordan trigger circuit as the first electronic latch, later the basic storage element of digital circuitry.

László Fejes Tóth (1953). Lagerungen in der Ebene, auf der Kugel und im Raum. Berlin: Springer.

László Fejes Tóth (1964). Regular Figures. Oxford: Pergamon Press.

Jean-Christophe Filliâtre, Léon Gondelman and Andrei Paskevich (2014). The spirit of ghost code. Computer Aided Verification (CAV 2014), Lecture Notes in Computer Science 8559, pp. 1–16. DOI: 10.1007/978-3-319-08867-9_1.

Nicholas Griffin, ed. (2003). The Cambridge Companion to Bertrand Russell. Cambridge University Press. G. G. Berry as a junior librarian at the Bodleian, p. 63.

Jean-Pierre Marquis (1996, substantive revision 2019). Category Theory. Stanford Encyclopedia of Philosophy. Entry. The borrowed words “category” and “functor”.

Ross M. McConnell, Kurt Mehlhorn, Stefan Näher and Pascal Schweitzer (2011). Certifying algorithms. Computer Science Review 5(2), pp. 119–161. DOI: 10.1016/j.cosrev.2010.09.009.

Jason Megill. The Lucas-Penrose Argument about Gödel's Theorem. Internet Encyclopedia of Philosophy. Article.

Glen Mével, Jacques-Henri Jourdan and François Pottier (2019). Time credits and time receipts in Iris. ESOP 2019, LNCS 11423, pp. 3–29. DOI: 10.1007/978-3-030-17184-1_1.

Jeff Miller. Earliest Uses of Symbols of Set Theory and Logic. MacTutor History of Mathematics. Page. Peano's epsilon for *est*, 1889.

J. J. O'Connor and E. F. Robertson (n.d.). The four colour theorem. MacTutor History of Mathematics. Entry. (De Morgan's letter to Hamilton of 23 October 1852 is quoted from here.)

Panu Raatikainen (2013, revised 2025). Gödel's Incompleteness Theorems. Stanford Encyclopedia of Philosophy. Entry. Source for the cautions about what the Gödel sentence says.

Erich Reck (2008, substantive revision 2025). Dedekind's Contributions to the Foundations of Mathematics. Stanford Encyclopedia of Philosophy. Entry. Peano acknowledged Dedekind's priority for the axioms.

Neil Robertson, Daniel Sanders, Paul Seymour and Robin Thomas (1997). The four-colour theorem. Journal of Combinatorial Theory, Series B 70(1), pp. 2–44.

Jürgen Schmidhuber (2003). Gödel machines: self-referential universal problem solvers making provably optimal self-improvements. Technical Report IDSIA-19-03. arXiv: cs/0309048.

Marcel-Paul Schützenberger (1961). On the definition of a family of automata. Information and Control 4(2–3), pp. 245–270.

Dana Scott (1962). A system of functional abstraction. Lectures delivered at the University of California, Berkeley, 1962–63; unpublished. The origin of the Scott encoding, as given in Mogensen–Scott encoding, Wikipedia, Article.

Dana Scott. Carnegie Mellon University's School of Computer Science, faculty Turing Award recipients: the 1976 award jointly with Michael Rabin for their work on nondeterministic machines. His doctorate at Princeton under Alonzo Church, domain theory and models of the lambda calculus: Wikipedia, Dana Scott.

Emin Gün Sirer, Willem de Bruijn, Patrick Reynolds, Alan Shieh, Kevin Walsh, Dan Williams and Fred B. Schneider (2011). Logical attestation: an authoriza-

tion architecture for trustworthy computing. SOSP 2011, pp. 249–264. DOI: 10.1145/2043556.2043580.

Ken Thompson. Heidelberg Laureate Forum, laureate profile: the 1983 Turing Award with Dennis Ritchie for operating systems theory and the implementation of Unix. Thompson's design and implementation of the original Unix at Bell Labs: Wikipedia, Ken Thompson.

Alfred North Whitehead and Bertrand Russell (1910). Principia Mathematica, vol. I. Cambridge University Press. Introduction, chapter II.

Robin Wilson (2002). Four Colours Suffice? Gresham College lecture, 21 October 2002. Lecture. De Morgan's letter of 23 October 1852 to the Irish mathematician William Rowan Hamilton.

18.4 Environmental records

M.-C. Chen et al. (2019). Emergence of classical objectivity of quantum Darwinism in a photonic quantum simulator. Science Bulletin 64, p. 580.

M. A. Ciampini, G. Pinna, P. Mataloni and M. Paternostro (2018). Experimental signature of quantum Darwinism in photonic cluster states. Physical Review A 98, 020101.

Mauro Dorato and Jan Faye (2026). Niels Bohr as a Physicist of Principle. arXiv: 2609.04297. Quotes Bohr's Unity of Knowledge (1954) on irreversible amplification and permanent marks on a photographic plate, citing p. 73.

Erwin Schrödinger. Wikipedia. Article. Austrian physicist; the wave equation of 1926.

T. Unden, D. Louzon, M. Zwolak, W. H. Zurek and F. Jelezko (2019). Revealing the emergence of classicality using nitrogen-vacancy centers. Physical Review Letters 123, 140402.

COLOPHON

This book is set in Charter, drawn by Matthew Carter in 1987 for the laser printers of its day, 10.5 on 14 point, on a page 189 by 246 millimetres. It was built from its source with pdfTeX. The cells are drawn in TikZ.

Its own count, taken by the build: 22 numbered statements, of which 22 are closed by a proof on the page, and the rest stated and not proved: . One cell for each, in the order they come, filled where the proof closed:



